



MEMORIA SISTEMAS DISTRIBUIDOS PL2-G14



SD-PL2-G14
Adrián Manso y Yonathan Patricio

2025-2026

INDICE DE TODAS LAS PRACTICAS:

PRACTICA 1: COMUNICACIÓN CON SOCKETS MEMOR-UDP Y CONCURRENTES PARALELA Y ASÍNCRONA	4
1. Parte 1: Comunicación con Sockets UDP	5
EJ1: Protocolo UDP:	5
EJ2: Pérdida de paquetes:	5
EJ3: Detección pérdida de paquetes	6
EJ4: Reenviar paquetes perdidos:	6
EJ6: Servicios con Broadcast	7
5. DESPLEGAR EN DOCKER	7
-Conclusiones de la semana 1	8
-Participación	8
2. Parte 2: Comunicación con Sockets TCP	9
EJ1: Mensajes de Tamaño Fijo.	9
EJ2: Mensajes de tamaño fijo (sendall y recvall)	10
EJ3: Delimitación de mensajes “\r\n”	10
EJ4: Lectura con Readline().	11
-EJ5: Enviar previamente la longitud del mensaje.	11
Despliegue en Docker:	11
Experimento:	12
Conclusiones de la parte 2.	13
-Participación	13
3. Parte 3: Concurrencia y Paralelismo.	14
EJ1: Servidor Secuencial de cálculo de primos.	14
EJ2: Servidor con hilos(threading)	14
EJ3: Servidor con Procesos (fork())	15
Conclusiones Parte 3.	16
-Participación	16
PRACTICA 2: IMPLEMENTACION DE UNA APLICACIÓN DE MENSAJERIA PEER-TO-PEER	17
1. Introducción:	18
2. Parte 1: Implementación de Chat P2P Simple	18
2.1 Introducción Chat Simple	18
2.2 Ejercicio 1: chat P2P	18
2.3 Ejercicio 1: con máquinas distintas	19
3. Parte 2: Mejora con Redis	19
3.1 Introducción Redis	19
3.2 Ejercicio 1: chat con Redis	19
3.3 Ejercicio 2: Gestión de claves y borrado manual	20
4. Conclusiones	21
5.Participación	21

PRACTICA 3: WEB SCRAPPING	22
1. Introducción	23
2. Parte 1: Scraping con Requests, BeautifulSoup y Pandas	23
2.1 Ejercicio 1: Extracción de puntuación en FilmAffinity	23
2.2 Ejercicio 2: Clasificación de Tipos (Películas y Series)	24
2.3 Ejercicio 3: Scraping a 1º FEB de Baloncesto:	24
3. Parte 2: Scraping con Scrapy	26
3.1 Ejercicio 4: Extracción de directores con selectores	26
3.2 Ejercicio 5: Spider personalizado con selectores CSS y XPath	27
4. Conclusiones	29
5. Participación	29
PRACTICA 4: APLICACIÓN RESTFUL DE GESTIÓN DE TAREAS	30
1. Introducción	32
2. (Parte 1): introducción a Flash y Unicorn.	32
2.1 Ejecutar entorno y Hola Mundo con Flask.	32
Ejercicio 1 “Hola Mundo”:	32
2.2 Otras Formas de conectividad	33
Ejercicio 2: Utilizando CURL	33
Ejercicio 3: Utilizando HTTPie.	34
2.3 Servidores WSGI.	35
3. (Parte 2): Desarrollo de la aplicación Restful de tareas.	35
3.1 Introducción:	35
3.2 Implementación:	35
3.2.1. GET /lista/v1/tareas (Experimento 1)	35
3.2.2: GET /lista/v1/tarea/<id>	36
3.3: POST /lista/v1/tareas	38
3.4: PUT /lista/v1/tarea/<id>	41
3.5.5 DELETE /lista/v1/tarea/<id>	41
3.6: Nivel 3: HATEOAS	44
4. (Parte 3) Clientes para la aplicación To-Do.	46
4.1: Cliente mediante interfaz gráfica	46
Ejercicio 1 Modificación hateos:	46
4.2: Políticas CORS	46
Ejercicio 2 Política CORS:	46
4.3. Cliente mediante línea de comandos	48
Ejercicio 3 TO-DO cliente click:	48
5. (Parte 4): Aplicación To-Do (BBDD y Autenticación)	49
5.1 Base de Datos	49
Ejercicio 1 Implementamos la Base de Datos:	49
Ejercicio 2: Probamos con 4 workers.	51
5.2 Seguridad basada en Autenticación	52
5.2.2 Implementación de Seguridad Básica (HTTP Auth) y Hash.	52

6. (Parte 5) Despliegue en Docker.	55
6.1 Contenedor MariaDB.	55
6.2 Gunicorn + Flask.	55
Ejercicio 1 Configuramos conexión con la BBDD:	55
Ejercicio 2: Construimos la imagen de nuestra aplicación y lanzamos el servidor.	56
Ejercicio 3: Comprobación persistencia.	57
6.3 Contenedor nginx (servidor web)	58
Ejercicio 4: Probando Funcionamiento:	58
Ejercicio 5: Imagen Infraestructura:	61
6.4 Automatizar Despliegue.	61
7.CONCLUSIONES	62
8.PARTICIPACIÓN	62
<i>PRACTICA 5: DESPLIEGUE DE LA APLICACIÓN “FILTROS API”</i>	63
1. Introducción	64
2. Diseño y Desarrollo de la Aplicación (parte 1)	64
2.1. Protocolo de Comunicación (Redis)	64
2.2 Servidor de Procesamiento(servidor_proc.py):	65
2.3 Api REST y Base de Datos (api_flask.py)	66
2.4 Cliente CLI (cliente.py)	68
3. Despliegue e Infraestructura Docker (parte 2)	69
3.1. Contenerización	69
3.2. Proxy (Nginx)	69
3.3 Ejercicio 1 Despliegue de la aplicación Práctica 5:	69
4. Pruebas y Resultados	71
Prueba 1: Procesamiento Exitoso.	71
Prueba 2: Interfaz Web:	72
Prueba 3: Lanzar varios trabajos a la vez:	73
Prueba 5: Manejo de Errores.	76
Ejercicio 2. Desplegar practica 4.	77
5.Conclusión.	78
6.Participación.	78

PRACTICA 1: COMUNICACIÓN CON SOCKETS MEMOR-UDP Y CONCURRENTE PARALELA Y ASÍNCRONA

1. Parte 1: Comunicación con Sockets UDP 5

EJ1: Protocolo UDP:	5
EJ2: Pérdida de paquetes:	5
EJ3: Detección pérdida de paquetes	6
EJ4: Reenviar paquetes perdidos:	6
EJ6: Servicios con Broadcast	7
5. DESPLEGAR EN DOCKER.....	7
-Conclusiones de la semana 1	8
-Participación.....	8

2. Parte 2: Comunicación con Sockets TCP 9

EJ1: Mensajes de Tamaño Fijo.	9
EJ2: Mensajes de tamaño fijo (sendall y recvall)	10
EJ3: Delimitación de mensajes “\r\n”	10
EJ4: Lectura con Readline().	11
Despliegue en Docker:	11
Experimento:	12
-EJ5: Enviar previamente la longitud del mensaje.	¡Error! Marcador no definido.
Conclusiones de la Práctica 2	13
-Participación.....	13

3. Parte 3: Concurrencia y Paralelismo. 14

EJ1: Servidor Secuencial de cálculo de primos.	14
EJ2: Servidor con hilos(threading)	14
EJ3: Servidor con Procesos (fork())	15
Conclusiones Parte 3	16
-Participación.....	16

1. Parte 1: Comunicación con Sockets UDP

En esta primera práctica aprendimos los fundamentos de la comunicación en red utilizando el protocolo UDP. El objetivo principal fue comprender su naturaleza no orientada a conexión y no fiable, e implementar mecanismos como la pérdida de paquetes. Finalmente, se utilizó la capacidad de broadcast de UDP para implementar un sistema de descubrimiento de servicios en una red local, desplegando la solución sobre contenedores Docker.

EJ1: Protocolo UDP:

Se implementó un servidor UDP básico que escucha en un puerto y muestra los mensajes recibidos, junto con la dirección de origen. El cliente es capaz de enviar mensajes a dicho servidor.

En las capturas adjuntas se observa: el servidor (izquierda) recibe los mensajes que el cliente (derecha) envía.

```

CTraceback (most recent call last):
  File "/home/uo300480/sd-pl2-g14/P1/EJ1/udp_servidor1.py", line 19, in <module>
    mensaje, dir=sock.recvfrom(1024)
KeyboardInterrupt

uo300480@sd-300480:~/sd-pl2-g14/P1/EJ1$ python3 udp_servidor.py
python3: can't open file '/home/uo300480/sd-pl2-g14/P1/EJ1/udp_servidor.py': [Errno 2] No such file or directory
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ1$ python3 udp_servidor1.py
Servidor UDP escuchando en el puerto 9999
Mensaje recibido de ('127.0.0.1', 50078): wfegfe

uo300480@sd-300480:~/sd-pl2-g14/P1/EJ1$ python3 udp_cliente1.py
Introduce un mensaje para enviar (o 'FIN' para terminar): dkkd
Introduce un mensaje para enviar (o 'FIN' para terminar): dgdg
Introduce un mensaje para enviar (o 'FIN' para terminar): FIN
Cerrando cliente...
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ1$ nc -u localhost 9999
wfegfe
```

EJ2: Pérdida de paquetes:

Se alteró el servidor para que, de forma aleatoria con una probabilidad del 50%, descarte los datagramas recibidos, simulando así una pérdida de paquetes en la red.

En las imágenes adjuntas se puede ver en la consola del servidor (izquierda) la secuencia de mensajes 'HOLA' enviados y los paquetes que se habían "perdido"

```

KeyboardInterrupt

uo300480@sd-300480:~/sd-pl2-g14/P1/EJ1$ ls
udp_cliente1.py  udp_servidor1.py
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ1$ cd ..
uo300480@sd-300480:~/sd-pl2-g14/P1$ cd EJ2
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ2$ ls
udp_cliente2_numero_mensajes.py  udp_servidor2_simula_perdidas.py
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ2$ python3 "C
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ2$ python3 udp_servidor2_simula_perdidas.py
Servidor UDP (simula perdidas) escuchando en el puerto 9999
Mensaje recibido de ('127.0.0.1', 38987): 1: ikgdgdg
Mensaje recibido de ('127.0.0.1', 38987): 2: Hola
Mensaje recibido de ('127.0.0.1', 38987): 3: Hola
Simulando paquete perdido desde ('127.0.0.1', 38987)

4740b34..318793f main -> main
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ2$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ2$ python3 udp_cliente2_numero_mensajes.py
Introduce un mensaje para enviar (o 'FIN' para terminar): ikgdgdg
Enviando: '1: ikgdgdg'
Introduce un mensaje para enviar (o 'FIN' para terminar): Hola
Enviando: '2: Hola'
Introduce un mensaje para enviar (o 'FIN' para terminar): Hola
Enviando: '3: Hola'
Introduce un mensaje para enviar (o 'FIN' para terminar): Hola
Enviando: '4: Hola'
Introduce un mensaje para enviar (o 'FIN' para terminar):
```

EJ3: Detección pérdida de paquetes

Este servidor UDP está programado para ignorar aleatoriamente algunos de los mensajes que recibe para simular la pérdida de paquetes en una red.

El cliente envió los dos primeros mensajes con éxito y recibiendo su confirmación. El cliente envió el tercer mensaje “adiós”, pero el servidor lo detectó como perdido, tal como se ve en su consola. Como el servidor nunca procesó ese mensaje, tampoco envió la confirmación que el cliente estaba esperando, y se mostró el error “No se recibió confirmación del servidor (timeout)”.

```
no301402$cd~no301402~/P1/S1/6J3$ python3 udp_servidor3_on_ek.py
Servidor UDP (con ek) escuchando en el puerto 9999
Mensaje recibido de (*127.0.0.1*, 53811): 1: hola
Mensaje recibido de (*127.0.0.1*, 53811): 2: hola
Simulando paquete perdido desde (*127.0.0.1*, 53811)

no301402$cd~no301402~/P1/S1/6J3$ python3 udp_cliente3_empra3_on_ek.py
python3 can't open file /home/usuario301402/P1/S1/6J3/udp_cliente3_empra3_on_ek.py: [Err
no 2] No such file or directory
no301402$cd~no301402~/P1/S1/6J3$ python3 udp_cliente3_espera_ek.py
Introduce un mensaje para enviar (o 'FIN' para terminar): hola
Enviando: '1: hola'
Recibida confirmación 'OK' del servidor.
Introduce un mensaje para enviar (o 'FIN' para terminar): hola
Enviando: '2: hola'
Recibida confirmación 'OK' del servidor.
Introduce un mensaje para enviar (o 'FIN' para terminar): adios
Enviando: '3: adios'
ERROR: No se recibió confirmación del servidor (timeout).
Introduce un mensaje para enviar (o 'FIN' para terminar): FIN
Cerrando el cliente.
no301402$cd~no301402~/P1/S1/6J3$
```

EJ4: Reenviar paquetes perdidos:

Para construir un protocolo fiable sobre UDP, se modificó el cliente para que, en caso de no recibir una confirmación ("OK") del servidor tras un tiempo (timeout), reenvíe el último mensaje. Este timeout se duplica en cada reintento hasta un máximo de 2 segundos.

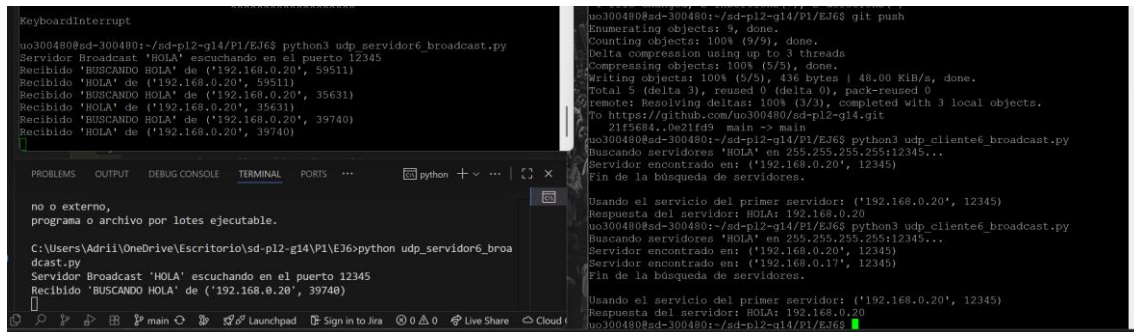
Cliente (izquierda): Se observa cómo, tras un Timeout, el cliente reintentará el envío del mensaje '1, Hola' con un timeout duplicado, hasta que finalmente recibe la confirmación del servidor.

Servidor (derecha): El servidor simula la pérdida de paquetes de forma aleatoria, lo que provoca los timeouts en el cliente. Cuando el servidor decide procesar el paquete, lo muestra por pantalla y envía la confirmación.

```
00300480@sd-300480:/sd-p12-g14/P1/EJ43 python3 udp_cliente4 reintera.py
Introduce un mensaje para enviar (o 'FIN' para terminar): Hola
Enviando (intento con timeout=0.1s): 'l, Hola'
Timeout. Reintentando...
Enviando (intento con timeout=0.2s): 'l, Hola'
Recibida confirmación 'OK' del servidor.
Enviando (intento con timeout=0.2s): 'l, Hola'
Timeout. Reintentando...
Enviando (intento con timeout=0.4s): 'l, Hola'
Recibida confirmación 'OK' del servidor.
Enviando (intento con timeout=0.4s): 'l, Hola'
Timeout. Reintentando...
Enviando (intento con timeout=0.8s): 'l, Hola'
Recibida confirmación 'OK' del servidor.
Enviando (intento con timeout=0.8s): 'l, Hola'
Timeout. Reintentando...
Enviando (intento con timeout=1.6s): 'l, Hola'
Timeout. Reintentando...
Puede q el server este caído, Intentelo mas tarde
Cerrando al cliente.
```

EJ6: Servicios con Broadcast

Se implementó un protocolo donde un cliente localiza servidores de un servicio "HOLA" sin conocer sus IPs previamente. El cliente envía un mensaje de *broadcast* ("BUSCANDO HOLA") a toda la subred, y responden múltiples servidores el de CMD en visual studio code y el de la máquina virtual.



```
KeyboardInterrupt
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ6$ python3 udp_servidor6_broadcast.py
Servidor Broadcast 'HOLA' escuchando en el puerto 12345
Recibido 'BUSCANDO HOLA' de ('192.168.0.20', 59511)
Recibido 'HOLA' de ('192.168.0.20', 59511)
Recibido 'BUSCANDO HOLA' de ('192.168.0.20', 35631)
Recibido 'HOLA' de ('192.168.0.20', 35631)
Recibido 'BUSCANDO HOLA' de ('192.168.0.20', 39740)
Recibido 'HOLA' de ('192.168.0.20', 39740)

no o externo,
programa o archivo por lotes ejecutable.

C:\Users\Adrii\OneDrive\Escritorio\sd-pl2-g14\P1\EJ6>python udp_servidor6_broa
dcast.py
Servidor Broadcast 'HOLA' escuchando en el puerto 12345
Recibido 'BUSCANDO HOLA' de ('192.168.0.20', 39740)

uo300480@sd-300480:~/sd-pl2-g14/P1/EJ6$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 3 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 436 bytes | 48.00 KiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/uo300480/sd-pl2-g14.git
21f5684..0e21f49 main -> main
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ6$ python3 udp_cliente6_broadcast.py
Buscando servidores 'HOLA' en 255.255.255:12345...
Servidor encontrado en: ('192.168.0.20', 12345)
Fin de la búsqueda de servidores.

Usando el servicio del primer servidor: ('192.168.0.20', 12345)
Respuesta del servidor: HOLA: 192.168.0.20
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ6$
```

5. DESPLEGAR EN DOCKER

Se creó una red Docker (pruebas) y se lanzaron tres contenedores con el servidor UDP.

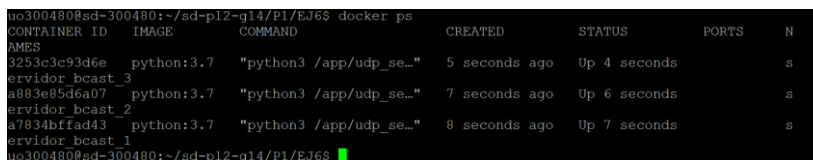
Se obtuvo la dirección de broadcast de la subred Docker (172.18.255.255).

Se lanzó el cliente en un cuarto contenedor, apuntando a la dirección de broadcast. Como se ve en la captura, el cliente descubre los tres servidores en ejecución (172.18.0.2, 172.18.0.3, 172.18.0.4), elige el primero y utiliza su servicio.

La respuesta del servidor, HOLA: 172.18.0.5, nos indica que la IP asignada al contenedor cliente es 172.18.0.5.

Instrucciones para desplegarlo en Docker:

1. Docker container prune
2. Docker network create pruebas



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	N
3253c3c93d6e	python:3.7	"python3 /app/udp_se..."	5 seconds ago	Up 4 seconds		s
a803e85d6a07	python:3.7	"python3 /app/udp_se..."	7 seconds ago	Up 6 seconds		s
a7834bffd43	python:3.7	"python3 /app/udp_se..."	8 seconds ago	Up 7 seconds		s

3. Averiguar dirección → Conseguimos la ip 172.18.0.2/16---→172.18.255.255

4. Lanzar cliente



```
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ6$ docker run -it --network
te6_broadcast.py 172.18.255.255 12345
Buscando servidores 'HOLA' en 172.18.255.255:12345...
Servidor encontrado en: ('172.18.0.2', 12345)
Servidor encontrado en: ('172.18.0.4', 12345)
Servidor encontrado en: ('172.18.0.3', 12345)
Fin de la búsqueda de servidores.

Usando el servicio del primer servidor: ('172.18.0.2', 12345)
Respuesta del servidor: HOLA: 172.18.0.5
uo300480@sd-300480:~/sd-pl2-g14/P1/EJ6$
```

-Conclusiones de la semana 1

Esta práctica ha servido para aprender sobre UDP. Para garantizar la entrega de mensajes, es imprescindible que la capa de aplicación implemente lógicas de numeración de secuencia, confirmaciones, *timeouts* y retransmisiones.

El uso de *broadcast* es una herramienta muy potente para la configuración dinámica y el descubrimiento de servicios en redes locales.

-Participación

Adrián Manso Martínez (70%): Se encargó de hacer parte del código y de las capturas probando los ejercicios.

Yonathan Patricio Torrejón Martínez (30%): Se encargó de probar los ejercicios y optimizar el código y ayudar en la memoria.

2. Parte 2: Comunicación con Sockets TCP

En esta segunda parte, usamos TCP (Transmission Control Protocol). A diferencia de UDP, TCP proporciona un canal de comunicación fiable y orientado a la conexión, presentándose como un flujo de bytes. El principal desafío abordado fue el problema del framing: cómo delimitar mensajes dentro de este flujo de bytes para que el receptor pueda procesarlos correctamente.

EJ1: Mensajes de Tamaño Fijo.

Aquí utilizamos mensajes de un tamaño prefijado y conocido tanto por el cliente como por el servidor (5 bytes en este caso).

El cliente envía mensajes de 5 bytes y el servidor los lee.

Servidor:

```
Esperando un cliente
Nuevo cliente conectado desde 127.0.0.1, 55872
Recibido mensaje: ABCDE
Recibido mensaje: ABCDE
Recibido mensaje: ABCDE
Recibido mensaje: ABCDE
Recibido mensaje: ABCDE
Recibido mensaje de finalización
Esperando un cliente
```

Cliente:

```
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ1$ python3 tcp_cliente1_simple.py
Conectado al servidor en localhost, 9999
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'FINAL'
Socket cerrado. Terminando cliente.
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ1$
```

Si ponemos ABCD en vez de ABCDE vemos que ocurre esto, el servidor no puede leer bloques de mensajes de bytes distintos a los fijados.

```
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ1$ python3 tcp_cliente1_simple.py localhost 8886
Conectado al servidor en localhost, 8886
Enviando: 'ABCD'
Enviando: 'ABCD'
Enviando: 'ABCD'
Enviando: 'ABCD'
Enviando: 'ABCD'
Enviando: 'FINAL'
Socket cerrado. Terminando cliente.
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ1$
```

```
Esperando un cliente
Nuevo cliente conectado desde 127.0.0.1, 53380
Recibido mensaje: ABCDA
Recibido mensaje: BCDAB
Recibido mensaje: CDABC
Recibido mensaje: DABCD
Recibido mensaje de finalización
Esperando un cliente
```

EJ2: Mensajes de tamaño fijo (sendall y recvall)

```
uo301402@sd-uo301402:~$ python3 tcp_servidor2_recvall.py
Servidor TCP (con recvall) escuchando en 9999
Esperando un cliente
Nuevo cliente conectado desde 127.0.0.1, 43062
Recibido mensaje de finalización
Esperando un cliente

uo301402@sd-uo301402:~$ python3 tcp_cliente2_sendall.py
Conectado al servidor en localhost, 9999
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'ABCDE'
Enviando: 'FINAL'
Socket cerrado. Terminando cliente.
uo301402@sd-uo301402:~$
```

El cliente se conecta al servidor y le envía mensajes de tamaño fijo. Utilizamos `sendall()` en lugar de `send()` para asegurar que cada mensaje de 5 bytes, "ABCDE", se envía por completo, repitiendo esto cinco veces. Después, envía un mensaje final de terminación, "FINAL", y cierra la conexión.

El servidor define una función "recvall" que garantiza la recepción del número exacto de bytes esperados. En su bucle principal, recibe los datos, y aunque no imprime los mensajes "ABCDE", reacciona correctamente al recibir "FINAL", e imprime el mensaje para terminar la conexión.

EJ3: Delimitación de mensajes "\r\n"

Se implementó un servicio "oche" (que devuelve los mensajes invertidos) donde cada mensaje, sin importar su longitud, finaliza con la secuencia de fin de línea `\r\n`

La terminal del cliente muestra cómo se envían tres líneas distintas (Hola, Mundo, Sistemas Distribuidos), cada una terminada con `\r\n`. Por cada una, se recibe la respuesta invertida del servidor, demostrando que este es capaz de leer y procesar cada mensaje de forma individual gracias al delimitador

En algunos ejercicios utilizamos `repr()` porque a veces cuando queremos representar un string en vez de bytes vemos que `print()` lo que hace es hacer un salto de línea en vez de mostrar `"\\n"` y la función `repr()` muestra la cadena como es `"\\r\\n"` evitando el salto de línea.

Servidor:

```
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ3$ python3 tcp_servidor3_oche_simplista.py
Servidor OCHE escuchando en el puerto 9999
Esperando un cliente
Nuevo cliente conectado desde 127.0.0.1, 52784
Conexión cerrada por el cliente.
Socket cerrado
Esperando un cliente
```

Cliente:

```
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ3$ python3 tcp_cliente3_oche.py
Conectado al servidor en localhost, 9999
'Hola\r\n'
Recibido: b'aloH\r\n'
'Mundo\r\n'
Recibido: b'odnuM\r\n'
'Sistemas Distribuidos\r\n'
Recibido: b'sodiubirtsID sametsiS\r\n'
Socket cerrado
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ3$
```

EJ4: Lectura con Readline().

Se convirtió el socket en un objeto similar a un fichero usando `socket.makefile()`. Esto nos permitió usar el método `f.readline()`, que lee del stream de datos hasta que encuentra el delimitador de fin de línea (`\r\n`), devolviendo un solo mensaje completo sin importar cómo se fragmentaron o agruparon los datos en la red.

El servidor sigue procesando cada línea de forma individual y correcta gracias a `readline()`. El cliente (derecha) envía tres mensajes y el servidor (izquierda) los recibe y procesa correctamente uno a uno

```
tcp_cliente5_oches_len.py tcp_servidor5_oches_len.py
uo300480@sd-300480:~/EJ4$ cd ..
uo300480@sd-300480:~/EJ4$ python3 tcp_servidor4_oches_readline.py
Servidor OCHES escuchando en el puerto 9999
Esperando un cliente
Nuevo cliente conectado desde 127.0.0.1, 51140
Conexión cerrada por el cliente.
Esperando un cliente

uo300480@sd-300480:~/EJ4$ python3 tcp_cliente4_oches_readline.py localhost 9999
Conectado al servidor en localhost, 9999
'Hola\r\n'
Recibido: 'aloH\r\n'
'Mundo\r\n'
Recibido: 'odnuM\r\n'
'Sistemas Distribuidos\r\n'
Recibido: 'sodiubirtsiD sametsiS\r\n'
Socket cerrado
uo300480@sd-300480:~/EJ4$
```

-EJ5: Enviar previamente la longitud del mensaje.

El protocolo consiste en que el emisor primero envía la longitud del mensaje (como una cadena de texto terminada en `\n`), y justo después envía el mensaje en sí. El receptor, con ayuda de la función `recibe_longitud`, lee primero esta cabecera para determinar el tamaño exacto del contenido que viene a continuación y, después, utiliza `recvall` para leer esa cantidad precisa de bytes, obteniendo el mensaje.

Por cada mensaje, se vería primero el envío/recepción de un número (la longitud) y a continuación el envío/recepción del mensaje correspondiente.

```
uo301402@sd-uo301402:~/EJ5$ python3 tcp_cliente5_oches_len.py
Conectado al servidor en localhost, 9999
Recibido: 'aloH'
Recibido: 'odnuM'
Recibido: 'sodiubirtsiD sametsiS'
Socket cerrado
uo301402@sd-uo301402:~/EJ5$

uo301402@sd-uo301402:~/EJ5$ python3 tcp_servidor5_oches_len.py
Servidor OCHES escuchando en el puerto 9999
Esperando un cliente
Nuevo cliente conectado
Recibido (4 bytes): Hola
Respuesta enviada (con longitud: 4)
Recibido (5 bytes): Mundo
Respuesta enviada (con longitud: 5)
Recibido (21 bytes): Sistemas Distribuidos
Respuesta enviada (con longitud: 21)
Conexión cerrada por el cliente.
Socket de datos cerrado.
Esperando un cliente
```

Despliegue en Docker:

En esta parte, se utilizó Docker para desplegar el servicio `oches`. Primero, se crearon dos contenedores (cliente y servidor) dentro de una misma red virtual de Docker “pruebas”

Lanzamos el contenedor del servidor con el nombre `servidor_oches`.

Y probamos el contenedor desde cliente, que se conecta exitosamente.

```
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ4$ docker run -d --rm -p 9090:9999 --net
work pruebas --name servidor_oches -v $(pwd):/app python:3.7 python3 /app/tcp_ser
vidor4_oches_readline.py
a3379f9a642b798d2c906f99341835c1d30d752a6fc2bedb93af57e41a843fcd
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ4$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS
a3379f9a642b   python:3.7    "python3 /app/tcp_se..." 4 seconds ago  Up 4 second
s
0.0.0.0:9090->9999/tcp, [::]:9090->9999/tcp  servidor_oches
uo300480@sd-300480:~/sd-pl2-g14/P1/S2/EJ4$
```

```

uo300480@sd-300480:~/sd-pl2-gl4/P1/S2/EJ4$ python3 tcp_cliente4_oche_readline.py
localhost 9090
Conectado al servidor en localhost, 9090
'Hola\r\n'
Recibido: 'aloH\r\n'
'Mundo\r\n'
Recibido: 'odnuM\r\n'
'Sistemas Distribuidos\r\n'
Recibido: 'sodiubirtsiD sametsiS\r\n'
Socket cerrado

```

Experimento:

Hacemos el servicio accesible desde fuera de la máquina anfitrión. Para ello, se volvió a lanzar el contenedor del servidor, pero esta vez utilizando la opción `-p 9090:9999`. Esto mapea el puerto 9090 de la máquina anfitrión al puerto 9999 dentro del contenedor, donde el servidor está escuchando.

`python3 tcp_cliente4_oche_readline.py otra_ip 9090`

Al probar con distinto portátil y enviarle el cliente (se pueden ver que son distintas IPs)

```

tcp_cliente4_oche_readline.py tcp_servidor4_oche_readline.py
uo300480@sd-02:~/P1/S2/EJ4$ python3 tcp_cliente4_oche_readline.py 192.168.207.99 9090
Conectado al servidor en 192.168.207.99, 9090
'Hola\r\n'
Recibido: 'aloH\r\n'
'Mundo\r\n'
Recibido: 'odnuM\r\n'
'Sistemas Distribuidos\r\n'
Recibido: 'sodiubirtsiD sametsiS\r\n'
Socket cerrado
uo300480@sd-02:~/P1/S2/EJ4$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:7c:0e:9d:da:5d txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

vethp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.207.151 netmask 255.255.255.0 broadcast 192.168.207.255
    inet6 fe80::a00:27ff:fe7c:bd5d prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:7c:bd:5d txqueuelen 1000 (Ethernet)
    RX packets 210887 bytes 282388288 (282.3 MB)
    TX packets 19234 bytes 1321664 (1.3 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Conclusiones de la parte 2.

Esta práctica ha sido fundamental para entender que, aunque TCP garantiza la entrega ordenada de los bytes, es responsabilidad del programador establecer un protocolo a nivel de aplicación para estructurar esos bytes en mensajes discretos. Se han explorado y comparado tres técnicas de framing: tamaño fijo, delimitadores y envío de longitud.

-Participación

Adrián Manso Martínez (60%): Se encargó de hacer parte del código y de las capturas.

Yonathan Patricio Torrejon Martinez(40%) : Se encargo de hacer parte del código y la memoria.

3. Parte 3: Concurrency y Paralelismo.

El objetivo de esta práctica fue analizar diferentes estrategias para gestionar múltiples clientes de forma simultánea. Se partió de un servidor secuencial que realiza una tarea intensiva en CPU (cálculo de números primos) para evidenciar sus limitaciones y estudiar posteriormente soluciones concurrentes y paralelas que permitan atender a varios usuarios a la vez

EJ1: Servidor Secuencial de cálculo de primos.

Se desarrolló un código para calcular los números primos en un servidor TCP que, para cada cliente, calcula una cantidad solicitada de números primos. Este servidor base procesa a los clientes de uno en uno: no acepta una nueva conexión hasta que ha terminado por completo con la anterior.

```
uo300480@sd-300480:~/S3/EJ1$ python3 ejercicio1_servidor_primos_base.py 7777
El servidor está listo para recibir conexiones
- Esperando cliente -
Conexión establecida desde: ('127.0.0.1', 57976)
Calculando los primeros 22 números primos...
Conexión cerrada con: ('127.0.0.1', 57976)
- Esperando cliente -

Se han calculado 15 de los 22 números primos solicitados
La hora actual es: 17:18:06.381632
Se han calculado 20 de los 22 números primos solicitados
La hora actual es: 17:18:06.510546
Se han calculado 20 de los 22 números primos solicitados
La hora actual es: 17:18:33.392419
Primos: (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79)
La hora actual es: 17:18:33.392913
FIN
uo300480@sd-300480:~/S3/EJ1$
```

EJ2: Servidor con hilos(threading)

El hilo principal se dedica exclusivamente a esperar y aceptar nuevas conexiones de clientes.

Por cada cliente que se conecta, el hilo principal crea y lanza un nuevo hilo de trabajo

Este nuevo hilo es el que se encarga de todo.

El hilo principal no espera a que los hilos de trabajo terminen después de lanzar uno vuelve a esperar por un cliente nuevo.

Servidor:

Se ve cómo el hilo principal acepta una conexión, lanza el "hilo 0" e inmediatamente vuelve a "esperando cliente", lo que le permite aceptar una segunda conexión y lanzar el "hilo 1" sin tener que esperar a que el primero termine.

```
uo300480@sd-300480:~/sd-pl2-g14/P1/S3/EJ2$ python3 ejercicio2_servidor_primos_threading.py 9763
El servidor está listo para recibir conexiones
- Hilo principal esperando cliente -
Conexión establecida desde: ('127.0.0.1', 58546)
Soy el hilo 0
- Hilo principal esperando cliente -
Conexión establecida desde: ('127.0.0.1', 34416)
Soy el hilo 1
- Hilo principal esperando cliente -
Conexión cerrada con: ('127.0.0.1', 58546)
Conexión cerrada con: ('127.0.0.1', 34416)
```

A la izquierda el cliente que calcula 20 números primos

A la derecha el otro cliente que calcula 16 números primos

```
Se han calculado 10 de los 20 números primos solicitados
La hora actual es: 11:42:29.219647
Se han calculado 15 de los 20 números primos solicitados
La hora actual es: 11:42:29.321751
Se han calculado 15 de los 20 números primos solicitados
La hora actual es: 11:42:29.982445
Se han calculado 15 de los 20 números primos solicitados
La hora actual es: 11:42:30.102079
Se han calculado 15 de los 20 números primos solicitados
La hora actual es: 11:42:30.381655
Se han calculado 15 de los 20 números primos solicitados
La hora actual es: 11:42:30.486939
Se han calculado 15 de los 20 números primos solicitados
La hora actual es: 11:43:10.531045
Primos:[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71]FIN

La hora actual es: 11:41:58.029867
Se han calculado 10 de los 16 números primos solicitados
La hora actual es: 11:42:24.585103
Se han calculado 15 de los 16 números primos solicitados
La hora actual es: 11:42:24.717402
Se han calculado 15 de los 16 números primos solicitados
La hora actual es: 11:42:25.975056
Se han calculado 15 de los 16 números primos solicitados
La hora actual es: 11:42:26.123073
Se han calculado 15 de los 16 números primos solicitados
La hora actual es: 11:42:26.440031
Se han calculado 15 de los 16 números primos solicitados
La hora actual es: 11:42:26.617384
Se han calculado 15 de los 16 números primos solicitados
La hora actual es: 11:42:36.348132
Primos:[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53]FIN
```

EJ3: Servidor con Procesos (fork())

La creación de procesos mediante la llamada al sistema `os.fork()`. Usamos un código similar al anterior intercambiando el número de hilos por el número de procesos.

El proceso padre se encarga de esperar y aceptar conexiones.

Por cada nueva conexión, el padre crea un proceso hijo idéntico a él.

El proceso hijo se encarga de atender al cliente, mientras que el proceso padre vuelve inmediatamente a esperar nuevas conexiones.

Servidor (izquierda): El proceso principal acepta una conexión, crea el "proceso 0" y, sin esperar, vuelve a su tarea de aceptar clientes, atendiendo a una segunda conexión y creando el "proceso 1".

Clientes(abajo y derecha): Hay dos clientes siendo atendidos de forma simultánea. El cálculo de primos para ambos clientes avanza en paralelo, cada uno en un proceso aislado.

```
uo301402@sd-uo301402:~$ python3 ejercicio3_servidor_primos_fork.py 9100
El servidor está listo para recibir conexiones
- Proceso principal esperando cliente -
Conexión establecida desde: ('127.0.0.1', 47608)
Soy el proceso 0
- Proceso principal esperando cliente -
Conexión establecida desde: ('127.0.0.1', 47616)
Soy el proceso 1
- Proceso principal esperando cliente -
Conexión cerrada con: ('127.0.0.1', 47616)
Conexión cerrada con: ('127.0.0.1', 47608)

La hora actual es: 11:44:21.485498
Se han calculado 10 de los 18 números primos solicitados
La hora actual es: 11:44:43.564394
Se han calculado 15 de los 18 números primos solicitados
La hora actual es: 11:44:43.664741
Se han calculado 15 de los 18 números primos solicitados
La hora actual es: 11:44:44.274098
Se han calculado 15 de los 18 números primos solicitados
La hora actual es: 11:44:44.375528
Se han calculado 15 de los 18 números primos solicitados
La hora actual es: 11:44:44.581317
Se han calculado 15 de los 18 números primos solicitados
La hora actual es: 11:44:44.683556
Se han calculado 15 de los 18 números primos solicitados
La hora actual es: 11:45:05.126084
Primos:[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61]FIN
uo301402@sd-uo301402:~$

La hora actual es: 11:44:11.691420
Se han calculado 5 de los 15 números primos solicitados
La hora actual es: 11:44:11.793078
Se han calculado 5 de los 15 números primos solicitados
La hora actual es: 11:44:23.957244
Se han calculado 10 de los 15 números primos solicitados
La hora actual es: 11:44:24.060332
Se han calculado 10 de los 15 números primos solicitados
La hora actual es: 11:44:47.210980
Primos:[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]FIN
uo301402@sd-uo301402:~$
```

Conclusiones Parte 3.

En esta parte aprendimos que a través del servidor secuencial no puede atender a múltiples clientes simultáneamente.

El paralelismo tanto con hilos (threading) como con procesos (fork), es una solución efectiva. Permiten que el servidor atienda a múltiples clientes de forma simultánea, aprovechando más el sistema.

-Participación

Adrián Manso Martínez (40%): Se encargó de hacer el ejercicio 1 y de las capturas en la memoria.

Yonathan Patricio Torrejon Martinez (60%) : Se encargó de hacer el resto de los ejercicios.

PRACTICA 2: IMPLEMENTACION DE UNA APLICACIÓN DE MENSAJERIA PEER-TO-PEER

1. Introducción:	18
2. Parte 1: Implementación de Chat P2P Simple	18
2.1 Introducción Chat Simple	18
2.2 Ejercicio 1: chat P2P	18
2.3 Ejercicio 1: con máquinas distintas	19
3. Parte 2: Mejora con Redis	19
3.1 Introducción Redis	19
3.2 Ejercicio 1: chat con Redis	19
3.3 Ejercicio 2: Gestión de claves y borrado manual	20
4. Conclusiones	21
5. Participación	21

1. Introducción:

En esta práctica desarrollamos una aplicación de mensajería siguiendo la arquitectura Peer-To-Peer descentralizada. Los clientes se comunican directamente. Dos fases: Implementación básica utilizando sockets UDP y una versión avanzada que integra Redis como almacén de datos en memoria.

2. Parte 1: Implementación de Chat P2P Simple

2.1 Introducción Chat Simple

El objetivo fue crear un cliente de chat que utiliza el protocolo UDP para el intercambio de mensajes. Select() nos permitió implementar un multiplexor de entrada que permite atender varias fuentes en este el teclado (sys.stdin.fileno()) y el socket UDP.

Código fuente ./P2/ejercicio1_chat_p2p.py

2.2 Ejercicio 1: chat P2P

Realizamos la prueba lanzando dos clientes en la misma máquina utilizando puertos distintos(8888 y 8889)

Forma de ejecutarlo:

```
python3 ejercicio1_chat_p2p.py 8888 "nombre-Adri"
```

```
python3 ejercicio1_chat_p2p.py 8889 "nombre-Quique"
```

Se observa que el usuario "Adri" inicia el chat en el puerto 8888 y utiliza el comando /CHAT para establecer conexión mediante [IP] [PUERTO] del otro usuario "Quique"

```
uo300480@sd-300480:~/sd-pl2-gl4/P2$ python3 ejercicio1_chat_p2p.py 8888 Adri
Cliente de chat iniciado. Escuchando en puerto 8888
Tu nombre es: Adri
/CHAT 127.0.0.1 8889
Hola
me recibes
> > > > Quique: Hola
> > Quique: Te recibo
```

```
uo300480@sd-300480:~/sd-pl2-gl4/P2$ python3 ejercicio1_chat_p2p.py 8889 Quique
Cliente de chat iniciado. Escuchando en puerto 8889
Tu nombre es: Quique
> Adri: Hola
> > Adri: me recibes
/CHAT 127.0.0.1 8888
Hola
Te recibo
```

2.3 Ejercicio 1: con máquinas distintas

Probamos a conectarnos en máquinas distintas dando la [IP] [puerto] de la maquina con la que queramos hablar.

```
uo300480@sd-300480:~/sd-pl2-g14/P2$ python3 ejercicio1_chat_p2p.py 8885 Adril
Cliente de chat iniciado. Escuchando en puerto 8885
Tu nombre es: Adril
/CHAT 10.130.196.40 5050
Hola me recibes
> > mesihola
> > mesite recibo
Adios
```

```
uo301402@sd-uo301402:~$ python3 ejercicio1_chat_p2p.py 5050 "mesi"
Cliente de chat iniciado. Escuchando en puerto 5050
Tu nombre es: mesi
> Adril: Hola me recibes
/CHAT 10.130.196.232 8885
> > hola
> te recibo
> > Adril: Adios
```

3. Parte 2: Mejora con Redis

3.1 Introducción Redis

Para mejorar la arquitectura utilizamos Redis, una base de datos clave-valor en memoria. Utilizamos este sistema clave-valor para almacenar el Nick y una cadena de texto "IP:puerto" asociada a él.

Para obtener la IP de la máquina en la que se ejecuta el cliente utilizamos un mecanismo para establecer una conexión UDP con otra IP pública bien conocida (una IP pública conocida) y después utilizar el método `getsockname()` para obtener la dirección IP origen.

3.2 Ejercicio 1: chat con Redis

Se mejoró el cliente anterior para que se conecte a Redis. En esta mejora los usuarios pueden encontrarse solamente con su nombre "Nick" sin necesidad de poner la IP:Puerto manualmente.

Código fuente: `/P2/S2/ejercicio1_cliente_redis.py`

Para Chatear: `python3 ejercicio1_cliente_redis.py 'nick' 127.0.0.1 (/CHAT 'nick')`

Para borrar : `python3 ejercicio2_borrar_nick_redis.py 127.0.0.1 'nick'`

Ejecutamos dos clientes desde distintas terminales y probamos la comunicación:

```
(redis) uo300480@sd-300480:~/P2/S2$ python3 ejercicio1_cliente_redis.py Jony 127.0.0.1
Cliente de chat iniciado. Escuchando en 10.91.69.232:50174
Tu nombre es: Jony
> adrian: Hola Jony
```

```
(redis) uo300480@sd-300480:~/P2/S2$ python3 ejercicio1_cliente_redis.py Adrian 127.0.0.1
Cliente de chat iniciado. Escuchando en 10.91.69.232:37656
Tu nombre es: Adrian
/CHAT jonyy
> Hablando con jonyy:
Hola
```

Utilizamos la herramienta redis-cli para inspeccionar las claves almacenadas. Se pueden ver las distintas KEYS* utilizadas.

```
uo300480@sd-300480:~/P2/S2$ source ~/redis/bin/activate
(redis) uo300480@sd-300480:~/P2/S2$ redis-cli
127.0.0.1:6379>
127.0.0.1:6379>
127.0.0.1:6379>
127.0.0.1:6379>
127.0.0.1:6379>
127.0.0.1:6379> KEYS *
1) "Jony"
2) "adrian"
127.0.0.1:6379>

(redis) uo300480@sd-300480:~/P2/S2$ redis-cli
127.0.0.1:6379> KEYS *
1) "jonyy"
2) "Jony"
3) "Adrian"
4) "adrian"
127.0.0.1:6379>
```

3.3 Ejercicio 2: Gestión de claves y borrado manual

Uno de los fallos era que cuando un cliente se cerraba (Ctrl+C) en lugar de usar /QUIT, se quedaba en estado “zombie” en Redis, no dejaba volver a conectarse usando el mismo Nick.

Para solucionar esto, hicimos el código (ejercicio2_borrar_nick_redis.py) que fuerza el borrado de la clave Nick usando el método delete() de Redis.

Nos intentamos conectar con el usuario "Luis", pero el sistema nos informa que ya está en uso. Ejecutamos el script para borrar el nick, que nos confirma que la clave ha sido eliminada. Ahora el usuario "Luis" logra conectarse de nuevo.

```
(redis) uo301402@sd-uo301402:~$ python3 ejercicio1_cliente_redis.py Luis 127.0.0.1
Cliente de chat iniciado. Escuchando en 10.91.69.40:59161
Tu nombre es: Luis
Ya esta en uso
(redis) uo301402@sd-uo301402:~$ python3 ejercicio2_borrar_nick_redis.py Luis 127.0.0.1
La clave ha sido eliminada
(redis) uo301402@sd-uo301402:~$ python3 ejercicio1_cliente_redis.py Luis 127.0.0.1
Cliente de chat iniciado. Escuchando en 10.91.69.40:50350
Tu nombre es: Luis
/QUIT
> Cerrando cliente
(redis) uo301402@sd-uo301402:~$ python3 ejercicio1_cliente_redis.py Luis 127.0.0.1
Cliente de chat iniciado. Escuchando en 10.91.69.40:50821
Tu nombre es: Luis
```

4. Conclusiones

Con la mejora de la parte 1 a la parte 2, aprendimos que utilizar Redis para los usuarios reduce la dificultad de conexión con otro cliente. El uso de UDP nos permitió tener una comunicación fácil, para chats en tiempo real, pero pueden perder los mensajes. Aprendimos que las claves no borradas pueden afectar al funcionamiento de la comunicación.

5.Participación

Adrián Manso:

P2(Parte 1) Desarrollo chat UDP: 50%

P2(Parte 2) Integración de Redis: 50%

P2(General) Redacción Memoria: 50%

Yonathan Patricio:

P2(Parte 1) Desarrollo chat UDP: 50%

P2(Parte 2) Integración de Redis: 50%

P2(General) Redacción Memoria: 50%

PRACTICA 3: WEB SCRAPPING

1. Introducción	23
2. Parte 1: Scraping con Requests, BeautifulSoup y Pandas	23
2.1 Ejercicio 1: Extracción de puntuación en FilmAffinity	23
2.2 Ejercicio 2: Clasificación de Tipos (Películas y Series)	24
2.3 Ejercicio 3: Scraping a 1º FEB de Baloncesto:	24
3. Parte 2: Scraping con Scrapy	26
3.1 Ejercicio 4: Extracción de directores con selectores	26
3.2 Ejercicio 5: Spider personalizado con selectores CSS y XPath	27
4. Conclusiones	29
5. Participación	29

1. Introducción

En esta práctica hemos trabajado con varias herramientas de Web Scraping, en la primera parte utilizando librerías de Python (Requests, BeautifulSoup, Pandas) y en la segunda parte utilizando Scrapy.

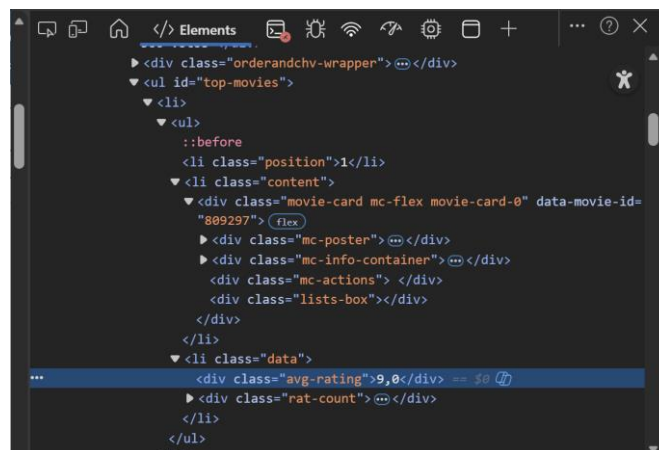
2. Parte 1: Scraping con Requests, BeautifulSoup y Pandas

En esta primera parte, utilizamos requests para obtener el HTML y BeautifulSoup para parsear el árbol DOM y navegar por sus etiquetas.

2.1 Ejercicio 1: Extracción de puntuación en FilmAffinity

El objetivo inicial fue ampliar el script base para obtener no solo el título y año, sino también la puntuación de los usuarios.

Análisis HTML: En la imagen se puede ver la inspección del HTML de la página de FilmAffinity, observamos que la puntuación no se encuentra dentro del contenedor principal div.movie-card que usábamos para el título. La puntuación estaba en el elemento hermano dentro de la lista, en un div con la clase avg-rating.



Se puede ver en la imagen como obtuvimos la puntuación.

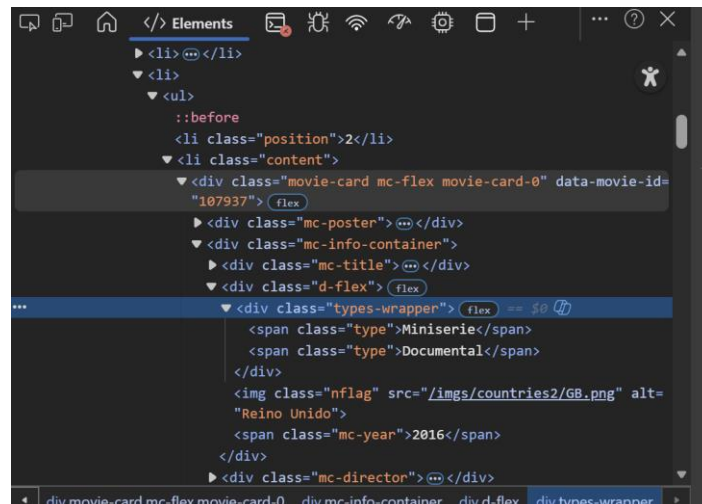
Código Fuente: ./P3

```
(venv) C:\Users\Adrii\OneDrive\Escritorio\Scrapping-FEBpython ejercicio1_find.py
El padrino (1972) Puntuacion: 9,0
Planeta Tierra II (2016) Puntuacion: 8,9
El padrino. Parte II (1974) Puntuacion: 8,9
```

2.2 Ejercicio 2: Clasificación de Tipos (Películas y Series)

FilmAffinity mezcla películas y series en sus listados. Mediante la inspección web, vimos que las series y miniseries incluyen etiquetas específicas (ver imagen).

Las películas no tenían estas etiquetas por lo que implementamos la lógica en el código utilizando el método `.find()` buscando estas clases. Si retornaba un objeto lo identificábamos como `Serie[S]`, en caso contrario (`None`) lo identificábamos como `Película [P]`.



Se puede ver la etiqueta correspondiente[P] o [S].

Código fuente: ./P3

```
(venv) C:\Users\Adria\OneDrive\Escritorio\Scrapping-FEB (venv) C:\Users\Adria\OneDrive\Escritorio\Scrapping-FEB>python ejercicio2_tipo.py
[P] El padrino (1972) Puntuacion: 9,0
[S] Planeta Tierra II (2016) Puntuacion: 8,9
[P] El padrino. Parte II (1974) Puntuacion: 8,9
```

2.3 Ejercicio 3: Scraping a 1º FEB de Baloncesto:

Para este ejercicio seleccionamos la página web de 1º FEB para extraer las estadísticas de la temporada regular (<https://www.primerafeb.com/estadisticas.aspx>)

El resultado fue extraer dos tablas “Medias por Equipo” y “Totales por Equipo”.

Inspección HTML: Localizamos las dos tablas y las filas correspondientes a cada Equipo. Iteramos sobre las filas de la tabla para extraer el nombre del equipo y algunas de sus estadísticas.

Utilizamos pandas para estructurar la información en un DataFrame y exportar a formatos CSV y Excel.

Código Fuente: ./P3/EJ3

2025/2026 Liga Regular Único

789.12 x 25

MEDIAS POR EQUIPO

EQUIPO	PAR	RCN	PT	T2	T3	TC	TL	REBOTES					TAPONES					FALTAS				
								RO	RD	RT	AS	BR	BP	TF	TC	MT	FC	FR	VA			
MELILLA CIUDAD DEL DEPORTE	6	200	74,2	49,3%	30%	41,5%	78,6%	10,2	25,3	35,5	14,3	5	16,2	1	1,7	2,7	20,5	22,2	74,2			
GRUPO ALEGA CANTABRIA	6	200	75,8	46,4%	34,9%	40,8%	76,4%	8,3	20,7	29	13,3	6,2	14,7	1,3	1,3	2	22	23,7	72,3			
GRUPO URETA TIZONA BURGOS	6	200	88,8	58,9%	27,8%	45,8%	77,9%	11,3	22,2	33,5	14,5	8,2	11,8	0,8	1,5	0,7	25,2	22,8	89,2			
HLA ALICANTE	5	200	84,6	53,1%	30,2%	46,8%	86,5%	9,8	22,8	32,6	15,8	6,4	12,2	1,4	3	0,6	21,8	20,6	91,8			
LEYMA CORUÑA	6	200	92,8	54,3%	34,1%	45,9%	72,7%	12,8	23,8	36,7	16	8,3	9,5	1,8	1,5	1,8	21,8	23,3	102,7			
ALMERKA OVIEDO BALONCESTO	6	200	80,5	57,7%	30,5%	44,4%	69,4%	9,8	24,8	34,7	15,5	6,8	16,2	2,2	2,7	1,8	20,8	21,3	82,2			

2025/2026 Liga Regular Único

td.tiros3 35.06 x 45

MEDIAS POR EQUIPO

EQUIPO	PAR	RCN	PT	T2	T3	TC	TL	REBOTES					TAPONES					FALTAS				
								RO	RD	RT	AS	BR	BP	TF	TC	MT	FC	FR	VA			
MELILLA CIUDAD DEL DEPORTE	6	200	74,2	49,3%	td.tiros3 35.06 x 45	25,3	35,5	14,3	5	16,2	1	1,7	2,7	20,5	21,2	74,2						
GRUPO ALEGA CANTABRIA	6	200	75,8	46,4%	34,9%	40,8%	76,4%	8,3	20,7	29	13,3	6,2	14,7	1,3	1,3	2	22	23,7	72,3			
GRUPO URETA TIZONA BURGOS	6	200	88,8	58,9%	27,8%	45,8%	77,9%	11,3	22,2	33,5	14,5	8,2	11,8	0,8	1,5	0,7	25,2	22,8	89,2			
HLA ALICANTE	5	200	84,6	53,1%	30,2%	46,8%	86,5%	9,8	22,8	32,6	15,8	6,4	12,2	1,4	3	0,6	21,8	20,6	91,8			
LEYMA CORUÑA	6	200	92,8	54,3%	34,1%	45,9%	72,7%	12,8	23,8	36,7	16	8,3	9,5	1,8	1,5	1,8	21,8	23,3	102,7			
ALMERKA OVIEDO BALONCESTO	6	200	80,5	57,7%	30,5%	44,4%	69,4%	9,8	24,8	34,7	15,5	6,8	16,2	2,2	2,7	1,8	20,8	21,3	82,2			
CAJA RURAL CB ZAMORA	6	200	91,5	54,6%	45%	51,7%	69,5%	9,8	24,7	34,5	14,7	6,7	16	3	1,8	2,8	20,8	21,8	97,2			

	Nombre	Puntos	Asistencia	Valoracion
0	MELILLA CIUDAD DEL DEPORTE	74.2	14.3	74.2
1	GRUPO ALEGA CANTABRIA	75.8	13.3	72.3
2	GRUPO URETA TIZONA BURGOS	88.8	14.5	89.2
3	HLA ALICANTE	84.6	15.8	91.8
4	LEYMA CORUÑA	92.8	16	102.7
5	ALJMERKA OVIEDO BALONCESTO	80.5	15.5	82.2
6	CAJA RURAL CB ZAMORA	91.5	14.7	97.2
7	INVEREADY GIPUZKOA	78.8	13.8	80.4
8	FIBWI MALLORCA BASQUET PALMA	76.8	17.8	75
9	PALMER BASKET MALLORCA PALMA	69.2	12.4	65.8
10	GRUPO CAESA SEGUROS FC CARTAGENA CB	70.8	10.7	64.7
11	HESTIA MENORCA	81.3	15.2	77.5
12	SÚPER AGROPAL PALENCIA	83	16.2	88.2
13	MONBUS OBRADOIRO	82	16.3	83.7
14	MOVISTAR ESTUDIANTES	89.5	14.5	97.3
15	CLUB OURENSE BALONCESTO	80.4	13.6	76.8
16	FLEXICAR FUENLABRADA	87.8	18.8	99.6

3. Parte 2: Scraping con Scrapy

En esta parte implementamos un spider de Scrapy, y aplicamos selectores CSS y XPATH para obtener el año y los directores. Primero identificamos las clases HTML, para luego poder obtener el año mediante el uso de `.get()` ya que es un valor único, y los directores que como pueden ser más de uno utilizamos el método `.getall()`.

Código Fuente: `./P3/fa_spider_ejercicio4.py`

3.1 Ejercicio 4: Extracción de directores con selectores

Inspeccionamos la página de FilmAffinity. Se identificaron los contenedores específicos para el año y el director dentro de `div.movie-card`

Extracción del Año: Al inspeccionar el elemento, se observó que el año de lanzamiento se encuentra encapsulado en una etiqueta (`span`) con la clase `mc-year`.

Extracción del Director: El contenedor con la clase `mc-director` contiene etiquetas `<a>` con los nombres de los directores. Dado que una película puede tener múltiples directores utilizamos el método `getall()`. Para obtener todos los directores.

Selectores CSS:

Se utilizó `div.mc-title a::text` para extraer el título.

Se utilizó `span.mc-year::text` para extraer el año.

Se utilizó `div.mc-director a::text` para seleccionar el texto de todos los enlaces dentro del contenedor de directores. Para luego obtenerlos con `getall()`.

```
El padrino (1972) - Director: ['Francis Ford Coppola']
El padrino. Parte II (1974) - Director: ['Francis Ford Coppola']
Doce hombres sin piedad (1957) - Director: ['Sidney Lumet']
La lista de Schindler (1993) - Director: ['Steven Spielberg']
Testigo de cargo (1957) - Director: ['Billy Wilder']
Cadena perpetua (1994) - Director: ['Frank Darabont']
Luces de la ciudad (1931) - Director: ['Charles Chaplin']
El gran dictador (1940) - Director: ['Charles Chaplin']
Pulp Fiction (1994) - Director: ['Quentin Tarantino']
Tiempos modernos (1936) - Director: ['Charles Chaplin']
El golpe (1973) - Director: ['George Roy Hill']
Ser o no ser (1942) - Director: ['Ernst Lubitsch']
Harakiri (1962) - Director: ['Masaki Kobayashi']
El crepúsculo de los dioses (1950) - Director: ['Billy Wilder']
La vida es bella (1997) - Director: ['Roberto Benigni']
Senderos de gloria (1957) - Director: ['Stanley Kubrick']
Eva al desnudo (1950) - Director: ['Joseph L. Mankiewicz']
El infierno del odio (1963) - Director: ['Akira Kurosawa']
Los siete samuráis (1954) - Director: ['Akira Kurosawa']
Perdición (1944) - Director: ['Billy Wilder']
El apartamento (1960) - Director: ['Billy Wilder']
Ciudad de Dios (2002) - Director: ['Fernando Meirelles', 'Kátia Lund']
Amanecer (1927) - Director: ['F.W. Murnau']
El chico (1921) - Director: ['Charles Chaplin']
Psicosis (1960) - Director: ['Alfred Hitchcock']
La evasión (1960) - Director: ['Jacques Becker']
La quimera del oro (1925) - Director: ['Charles Chaplin']
Uno de los nuestros (1990) - Director: ['Martin Scorsese']
Casablanca (1942) - Director: ['Michael Curtiz']
El hombre que mató a Liberty Valance (1962) - Director: ['John Ford']
```

Selectores XPATH:

Se utilizó `'./div[contains(@class, "mc-title")]/a/text()'` para extraer el título.

Se utilizó `'./span[contains(@class, "mc-year")]/text()'` para extraer el año.

Se utilizó `'./div[contains(@class, "mc-director")]/a/text()'` para seleccionar el texto de todos los enlaces dentro del contenedor de directores. Para luego obtenerlos con `getall()`.

```
El padrino (1972) - Director: ['Francis Ford Coppola']
El padrino. Parte II (1974) - Director: ['Francis Ford Coppola']
Doce hombres sin piedad (1957) - Director: ['Sidney Lumet']
La lista de Schindler (1993) - Director: ['Steven Spielberg']
Testigo de cargo (1957) - Director: ['Billy Wilder']
Cadena perpetua (1994) - Director: ['Frank Darabont']
Luces de la ciudad (1931) - Director: ['Charles Chaplin']
El gran dictador (1940) - Director: ['Charles Chaplin']
Pulp Fiction (1994) - Director: ['Quentin Tarantino']
Tiempos modernos (1936) - Director: ['Charles Chaplin']
El golpe (1973) - Director: ['George Roy Hill']
Ser o no ser (1942) - Director: ['Ernst Lubitsch']
Harakiri (1962) - Director: ['Masaki Kobayashi']
El crepúsculo de los dioses (1950) - Director: ['Billy Wilder']
La vida es bella (1997) - Director: ['Roberto Benigni']
Senderos de gloria (1957) - Director: ['Stanley Kubrick']
Eva al desnudo (1950) - Director: ['Joseph L. Mankiewicz']
El infierno del odio (1963) - Director: ['Akira Kurosawa']
Los siete samuráis (1954) - Director: ['Akira Kurosawa']
Perdición (1944) - Director: ['Billy Wilder']
El apartamento (1960) - Director: ['Billy Wilder']
Ciudad de Dios (2002) - Director: ['Fernando Meirelles', 'Kátia Lund']
Amanecer (1927) - Director: ['F.W. Murnau']
El chico (1921) - Director: ['Charles Chaplin']
Psicosis (1960) - Director: ['Alfred Hitchcock']
La evasión (1960) - Director: ['Jacques Becker']
La quimera del oro (1925) - Director: ['Charles Chaplin']
Uno de los nuestros (1990) - Director: ['Martin Scorsese']
Casablanca (1942) - Director: ['Michael Curtiz']
El hombre que mató a Liberty Valance (1962) - Director: ['John Ford']
```

3.2 Ejercicio 5: Spider personalizado con selectores CSS y XPath

Aplicamos Scrapy al sitio web de 1º FEB Baloncesto para comparar el desarrollo.

Creamos dos spiders:

CSS: Utilizando selectores CSS para navegar por el código de la tabla de estadísticas

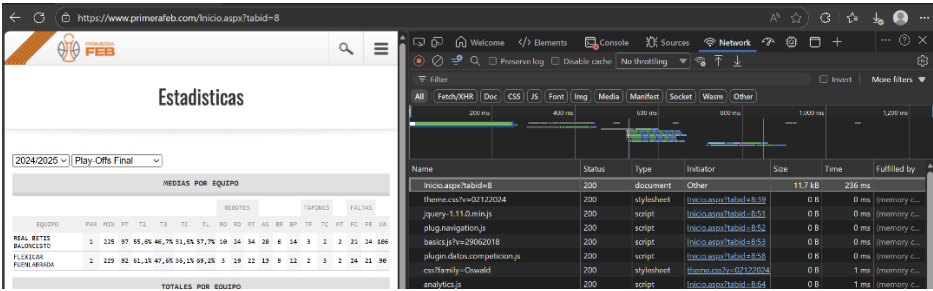
XPath: Utilizando rutas XML (XPath). Fue útil para navegar por elementos padre-hijo de la estructura de la página web.

Se extrajeron nuevas características de la tabla “Medias por Equipo” y utilizamos el mecanismo de Scrapy para generar el archivo CSV con la información obtenida.

Para obtener la temporada actual podemos ver la ruta de la pagina web:

```
<div class="wrap-select">
  <select name="_ctl0:temporadasDropDownList" onchange="javascript:setTimeout('__doPostBack(\'_ctl0$temporadasDropDownList\',\'\'), 0)"
    language="javascript" id="_ctl0_temporadasDropDownList">
    </select>
  </div>
  <select name="_ctl0:fasesGruposDropDownList" onchange="javascript:setTimeout('__doPostBack(\'_ctl0$fasesGruposDropDownList\',\'\'), 0)"
    language="javascript" id="_ctl0_fasesGruposDropDownList">
    </select>
</div>
```

Inspeccionamos la pestaña de “Red” para entender cómo se cargan los datos:



Código Fuente: ./P3/EJ5

Resultados CSV:

	1	Temporada	Nombre	Puntos	Asistencia	Valoracion	Triples	Mates
2		2025/2026	MELILLA CIUDAD DEL DEPORTE	74.9	15	76.1	29.2%	2.3
3		2025/2026	GRUPO ALEGA CANTABRIA	75.8	13.3	72.3	34.9%	2
4		2025/2026	GRUPO URETA TIZONA BURGOS	85.9	14.6	84.1	28.3%	0.7
5		2025/2026	HLA ALICANTE	85.5	16.8	94.2	39.1%	0.5
6		2025/2026	LEYMA CORUÑA	92.7	16.3	101.9	34.8%	1.7
7		2025/2026	ALIMERKA OVIEDO BALONCESTO	82.1	15.7	84.7	31.5%	1.7
8		2025/2026	CAJA RURAL CB ZAMORA	89.4	14.1	92.9	42.2%	3
9		2025/2026	INVEREADY GIPUZKOA	76.5	13.7	76.3	26.6%	3.2
10		2025/2026	FIBWI MALLORCA BASQUET PALMA	75.8	15.7	73.3	31.2%	1.8
11		2025/2026	PALMER BASKET MALLORCA PALMA	71.5	13	70.5	31%	1.8
12		2025/2026	GRUPO CAESA SEGUROS FC CARTAGENA CB	70	10.6	61.9	28.7%	1.1
13		2025/2026	HESTIA MENORCA	81.1	15.3	77.1	33.2%	1.7
14		2025/2026	SÚPER AGROPAL PALENCIA	85.7	16.1	91.1	32.3%	1.4
15		2025/2026	MONBUS OBRADOIRO	83.6	16.3	88.4	32.4%	2.4
16		2025/2026	MOVISTAR ESTUDIANTES	89.1	14.1	95.9	37.1%	2.6
17		2025/2026	CLUB OURENSE BALONCESTO	82.2	13.7	82.8	35.6%	2.2
18		2025/2026	FLEXICAR FUENLABRADA	85	17.7	92.7	41%	2.2

4. Conclusiones

Hemos comprobado que el scraping depende de la estructura de código de la web. Un pequeño cambio en el HTML o una protección de cookies puede romper el scraper.

Requests + BS4: Es fácil y se usa para scraping rápido, intuitivo.

Scrapy: Es un poco más complejo y muy superior en muchas funciones.

Aprendimos a inspeccionar la pestaña de "Red" (Network) del navegador para entender cómo se cargan los datos.

5. Participación

Adrián Manso:

P3(Parte 1) Requests/BS4: 50%

P3(Parte 2) Scrapy Spiders: 50%

P3(general) Redacción Memoria: 50%

Yonathan Patricio:

P3(Parte 1) Requests/BS4: 50%

P3(Parte 2) Scrapy Spiders: 50%

P3(general) Redacción Memoria: 50%

PRACTICA 4: APLICACIÓN RESTFUL DE GESTIÓN DE TAREAS

1. Introducción	32
2. (Parte 1): introducción a Flask y Gunicorn.	32
2.1 Ejecutar entorno y Hola Mundo con Flask.	32
Ejercicio 1 “Hola Mundo”:	32
2.2 Otras Formas de conectividad	33
Ejercicio 2: Utilizando CURL	33
Ejercicio 3: Utilizando HTTPie.	34
2.3 Servidores WSGI.	35
3. (Parte 2): Desarrollo de la aplicación Restful de tareas.	35
3.1 Introducción:	35
3.2 Implementación:	35
3.2.1. GET /lista/v1/tareas (Experimento 1)	35
3.2.2: GET /lista/v1/tarea/<id>	36
3.3: POST /lista/v1/tareas	38
3.4: PUT /lista/v1/tarea/<id>	41
3.5.5 DELETE /lista/v1/tarea/<id>	41
3.6: Nivel 3: HATEOAS	44
4. (Parte 3) Clientes para la aplicación To-Do.	46
4.1: Cliente mediante interfaz gráfica	46
Ejercicio 1 Modificación hateos:	46
4.2: Políticas CORS	46
Ejercicio 2 Política CORS:	46
4.3. Cliente mediante línea de comandos	48
Ejercicio 3 TO-DO cliente click:	48

5. (Parte 4): Aplicación To-Do (BBDD y Autenticación)	49
5.1 Base de Datos	49
Ejercicio 1 Implementamos la Base de Datos:	49
Ejercicio 2: Probamos con 4 workers.	51
5.2 Seguridad basada en Autenticación	52
5.2.2 Implementación de Seguridad Básica (HTTP Auth) y Hash.	52
6. (Parte 5) Despliegue en Docker.	55
6.1 Contenedor MariaDB.	55
6.2 Gunicorn + Flask.	55
Ejercicio 1 Configuramos conexión con la BBDD:	55
Ejercicio 2: Construimos la imagen de nuestra aplicación y lanzamos el servidor.	56
Ejercicio 3: Comprobación persistencia.	57
6.3 Contenedor nginx (servidor web)	58
Ejercicio 4: Probando Funcionamiento:	58
Ejercicio 5: Imagen Infraestructura:	61
6.4 Automatizar Despliegue.	61
7.CONCLUSIONES	62
8.PARTICIPACIÓN	62

1. Introducción

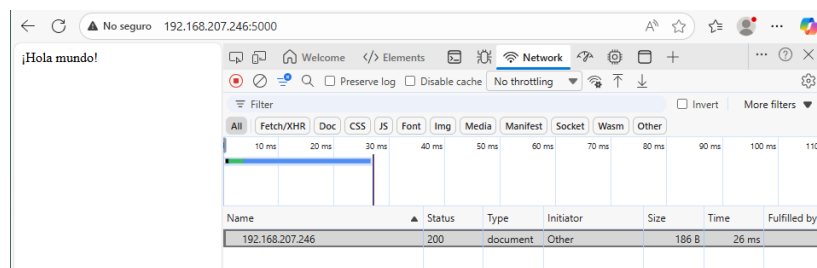
En esta práctica hemos abordado el ciclo completo de desarrollo y despliegue de una aplicación basada en servicios web RESTful. El objetivo ha sido crear una aplicación de gestión de tareas (*To-Do*) evolucionando desde un script básico en Flask hasta una arquitectura desplegada, incluyendo base de datos.

2. (Parte 1): introducción a Flash y Unicorn.

2.1 Ejecutar entorno y Hola Mundo con Flask.

Ejercicio 1 “Hola Mundo”:

Configuramos el entorno virtual y hacemos un Hola Mundo en Flask:



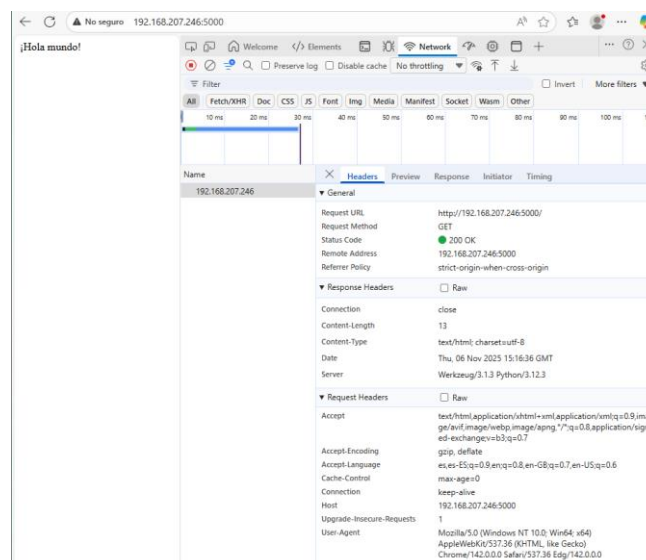
Verificamos el funcionamiento utilizando clientes HTTP. Flask genera las cabeceras HTTP, podemos ver lo que aparece en ellas:

Status Code: 200 OK, petición exitosa.

Content-length: 13 (bytes del “Hola Mundo”).

Content-Type: text/html; charset=utf-8, especifica el formato y codificación de la respuesta

Server: Werkzeug/3.1.3 Python/3.12.3 indica el servidor WSGI que utiliza Flask.



Salida en consola: En la terminal donde se ejecuta el servidor, observamos los logs de actividad. Primero, se indica que el servidor está corriendo en `http://192.168.207.246:5000`. Seguidamente, se registró la petición entrante con el formato: IP del cliente, fecha, comando (GET / HTTP/1.1) y el código de estado (200 OK)

```
(flaskenv) uo300480@sd-02:~/flaskenv$ FLASK_APP=hola_flask.py flask run --host=0.0.0.0
* Serving Flask app 'hola_flask.py'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.207.246:5000
Press CTRL+C to quit
http://192.168.207.246:5000192.168.207.211 - - [06/Nov/2025 15:13:19] "GET / HTTP/1.1" 200 -
192.168.207.211 - - [06/Nov/2025 15:13:19] "GET /favicon.ico HTTP/1.1" 404 -
192.168.207.211 - - [06/Nov/2025 15:14:05] "GET / HTTP/1.1" 200 -
192.168.207.211 - - [06/Nov/2025 15:16:06] "GET / HTTP/1.1" 200 -
http://127.0.0.1:5000http://192.168.207.246:5000192.168.207.211 - - [06/Nov/2025 15:16:29] "GET / HTTP/1.1" 200 -
192.168.207.211 - - [06/Nov/2025 15:16:36] "GET / HTTP/1.1" 200 -
```

2.2 Otras Formas de conectividad

Inspección de cabeceras HTTP mediante curl y HTTPie. Se observa el servidor WSGI Werkzeug utilizado.

Ejercicio 2: Utilizando CURL.

Para obtener la información del servidor hacemos la petición usando CURL, se puede ver como obtenemos los mismo que antes:

Probamos a hacer la petición a un sitio que no existe, para simular un error 404 Not Found.

```
uo301402@sd-uo301402: ~
uo301402@sd-uo301402:~$ curl -D- http://10.62.124.40:5000/
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.10.12
Date: Thu, 06 Nov 2025 15:50:05 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 13
Connection: close

¡Hola mundo!uo301402@sd-o>/ curl -D- http://10.62.124.40:5000/prueba
-bash: /: Is a directory
uo301402@sd-uo301402:~$ curl -D- http://10.62.124.40:5000/prueba
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.3 Python/3.10.12
Date: Thu, 06 Nov 2025 15:51:04 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 207
Connection: close

<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.</p>
uo301402@sd-uo301402:~$
```

Salida en la consola del servidor: En la terminal donde se ejecuta Flask, se registraron ambas interacciones mostrando la IP de origen, la fecha, el verbo, el recurso y el código de estado. Petición exitosa y la petición de error:

```
(flaskenv) uo301402@sd-uo301402:~$ FLASK_APP=hola_flask.py flask run --host=0.0.0.0
* Serving Flask app 'hola_flask.py'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.62.124.40:5000
Press CTRL+C to quit
10.62.124.40 - - [06/Nov/2025 15:50:05] "GET / HTTP/1.1" 200 -
10.62.124.40 - - [06/Nov/2025 15:51:04] "GET /prueba HTTP/1.1" 404 -
```

Ejercicio 3: Utilizando HTTPie.

Utilizamos HTTPie para hacer las peticiones.

No existen diferencias funcionales en la respuesta de los tres comandos, todos devolvieron un código 200 OK y el cuerpo "¡Hola mundo!".

La diferencia entre el primero y el segundo es solo la dirección de destino (localhost / IP)

Y la diferencia en el último es que HTTPie actúa igual poniendo el http GET <> que solo poniendo http <>.

```
(flaskenv) uo300480@sd-02:~$ http http://localhost:5000/
HTTP/1.1 200 OK
Connection: close
Content-Length: 13
Content-Type: text/html; charset=utf-8
Date: Thu, 06 Nov 2025 15:21:25 GMT
Server: Werkzeug/3.1.3 Python/3.12.3

¡Hola mundo!
```

```
(flaskenv) uo300480@sd-02:~$ http 192.168.207.246:5000/
HTTP/1.1 200 OK
Connection: close
Content-Length: 13
Content-Type: text/html; charset=utf-8
Date: Thu, 06 Nov 2025 15:22:05 GMT
Server: Werkzeug/3.1.3 Python/3.12.3

¡Hola mundo!
```

```
(flaskenv) uo300480@sd-02:~$ http GET http://192.168.207.246:5000/
HTTP/1.1 200 OK
Connection: close
Content-Length: 13
Content-Type: text/html; charset=utf-8
Date: Thu, 06 Nov 2025 15:22:41 GMT
Server: Werkzeug/3.1.3 Python/3.12.3

¡Hola mundo!
```

Ejecutamos el comando http POST <>, intentamos enviar datos al servidor, pero nos da un error 405 Method Not Allowed, esto pasa porque todavía no definimos el método POST en la aplicación Flask, pero el método GET ya venía definido por Flask.

```
(flaskenv) uo300480@sd-02:~$ http POST http://192.168.207.246:5000/
HTTP/1.1 405 METHOD NOT ALLOWED
Allow: HEAD, OPTIONS, GET
Connection: close
Content-Length: 153
Content-Type: text/html; charset=utf-8
Date: Thu, 06 Nov 2025 15:24:19 GMT
Server: Werkzeug/3.1.3 Python/3.12.3

<!doctype html>
<html lang=en>
<title>405 Method Not Allowed</title>
<h1>Method Not Allowed</h1>
<p>The method is not allowed for the requested URL.</p>
```

2.3 Servidores WSGI.

Sustituimos el servidor por Gunicorn capaz de manejar workers. Comprobamos otra vez la conexión de este nuevo servidor con CURL Y HTTPie.

```
;Hola mundo!(flaskenv) uo300480@sd-02:~$ curl http://0.0.0.0:5000
;Hola mundo!(flaskenv) uo300480@sd-02:~$ curl -D- http://0.0.0.0:5000
HTTP/1.1 200 OK
Server: gunicorn
Date: Thu, 06 Nov 2025 15:30:50 GMT
Connection: close
Content-Type: text/html; charset=utf-8
Content-Length: 13

;Hola mundo!(flaskenv) uo300480@sd-02:~$ http http://0.0.0.0:5000
HTTP/1.1 200 OK
Connection: close
Content-Length: 13
Content-Type: text/html; charset=utf-8
Date: Thu, 06 Nov 2025 15:32:30 GMT
Server: gunicorn

;Hola mundo!
```

3. (Parte 2): Desarrollo de la aplicación Restful de tareas.

3.1 Introducción:

Diseñamos la API siguiendo el modelo de madurez de Richardson. Definimos los recursos `/lista/v1/tareas` y `/lista/v1/tarea/<id>`.

GET: Lectura de tareas.

POST: Creación de tareas

PUT: Modificación de estado/descripción.

DELETE: Eliminación de recursos.

3.2 Implementación:

3.2.1. GET /lista/v1/tareas (Experimento 1)

Experimento 1: Implementación de GET.

Creamos el fichero `experimento1.py` (Código Fuente: `./P4/S1`) definiendo una lista global `tareas` y un endpoint a la ruta `/lista/v1/tareas`. Utilizamos la función `jsonify` de Flask para convertir la lista de Python en una respuesta JSON válida.

Probamos la conexión utilizando HTTPie y CURL para la petición GET:

Usando http: http GET <http://localhost:5000/lista/v1/tareas> vemos que nos devuelve el 200 OK, más cabeceras y el objeto JSON lista de tareas.

Usando curl: curl http://localhost:5000/lista/v1/tareas a diferencia de http, curl solo nos devuelve el objeto JSON de la lista de tareas.

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http GET http://localhost:5000/lista/v1/tareas
HTTP/1.1 200 OK
Connection: close
Content-Length: 14
Content-Type: application/json
Date: Sat, 15 Nov 2025 13:51:16 GMT
Server: gunicorn

{
  "tareas": []
}

(flaskenv) uo300480@sd-300480:~/TO-DO$ curl http://localhost:5000/lista/v1/tareas
{"tareas":[]}
(flaskenv) uo300480@sd-300480:~/TO-DO$
```

Probamos usando POST y nos da error 405, ya que el método aun no es válido, en la cabecera Allow nos dice que métodos son válidos.

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http POST http://localhost:5000/lista/v1/tareas
HTTP/1.1 405 METHOD NOT ALLOWED
Allow: GET, OPTIONS, HEAD
Connection: close
Content-Length: 153
Content-Type: text/html; charset=utf-8
Date: Sat, 15 Nov 2025 13:53:40 GMT
Server: gunicorn

<!doctype html>
<html lang=en>
<title>405 Method Not Allowed</title>
<h1>Method Not Allowed</h1>
<p>The method is not allowed for the requested URL.</p>
```

3.2.2: GET /lista/v1/tarea/<id>

Experimento 2 usando CURL y HTTPie:

Creamos el fichero experimento2.py (Código Fuente: ./P4/S1) y modificamos la lista de tareas añadiendo una lista de diccionarios predefinida.

Lanzamos el servidor y hacemos una petición GET con HTTP y CURL, para obtener la lista

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http GET http://localhost:5000/lista/v1/tareas
HTTP/1.1 200 OK
Connection: close
Content-Length: 189
Content-Type: application/json
Date: Sat, 15 Nov 2025 13:56:27 GMT
Server: gunicorn

{
  "tareas": [
    {
      "completada": true,
      "descripcion": "Terminar práctica Hola Mundo con Flask",
      "id": 1
    },
    {
      "completada": false,
      "descripcion": "Terminar práctica aplicación To-Do",
      "id": 2
    }
  ]
}
```

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ curl http://localhost:5000/lista/v1/tareas
{"tareas":[{"completada":true,"descripcion":"Terminar pr\u00e1ctica Hola Mundo con Flask","id":1},{"completada":false,"descripcion":"Terminar pr\u00e1ctica aplicaci\u00f3n To-Do","id":2}]}
(flaskenv) uo300480@sd-300480:~/TO-DO$
```

Ejercicio 1 Agregamos get tarea():

Creamos el fichero ejercicio1_get_tarea.py (código fuente: ./P4/S1) añadiendo la ruta /lista/v1/tarea/<int:id> para obtener una tarea.

Realizamos una petición GET para obtener las tareas con id: 1, 2 y 3:

```
(flaskenv) uo301402@sd-uo301402:~$ http GET http://localhost:5000/lista/v1/tarea/1
HTTP/1.1 200 OK
Connection: close
Content-Length: 97
Content-Type: application/json
Date: Mon, 17 Nov 2025 12:24:46 GMT
Server: gunicorn
{
  "tarea": {
    "completada": true,
    "descripcion": "Terminar práctica Hola Mundo con Flask",
    "id": 1
  }
}

(flaskenv) uo301402@sd-uo301402:~$ http GET http://localhost:5000/lista/v1/tarea/2
HTTP/1.1 200 OK
Connection: close
Content-Length: 99
Content-Type: application/json
Date: Mon, 17 Nov 2025 12:26:57 GMT
Server: gunicorn
{
  "tarea": {
    "completada": false,
    "descripcion": "Terminar práctica aplicación To-Do",
    "id": 2
  }
}

(flaskenv) uo301402@sd-uo301402:~$ http GET http://localhost:5000/lista/v1/tarea/3
HTTP/1.1 200 OK
Connection: close
Content-Length: 15
Content-Type: application/json
Date: Mon, 17 Nov 2025 12:27:24 GMT
Server: gunicorn
{
  "tarea": null
}
```

Ejercicio 2: Tarea inexistente:

Modificamos la función get_tarea(id) en (Código fuente: P4/S1/ejercicio2_get_tarea.py) para que, si se solicita una tarea inexistente, la aplicación devuelva un error 404. Para comprobarlo hacemos una petición http GET en la que primero obtenemos una tarea que existe y luego probamos a solicitar una que no existe y nos da error 404 Not Found.

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http GET http://localhost:5000/lista/v1/tarea/1
HTTP/1.1 200 OK
Connection: close
Content-Length: 97
Content-Type: application/json
Date: Mon, 17 Nov 2025 16:03:22 GMT
Server: gunicorn
{
  "tarea": {
    "completada": true,
    "descripcion": "Terminar práctica Hola Mundo con Flask",
    "id": 1
  }
}

(flaskenv) uo300480@sd-300480:~/TO-DO$ http GET http://localhost:5000/lista/v1/tarea/99
HTTP/1.1 404 NOT FOUND
Connection: close
Content-Length: 207
Content-Type: text/html; charset=utf-8
Date: Mon, 17 Nov 2025 16:05:15 GMT
Server: gunicorn
<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.</p>
```

Ejercicio 3 Manejador de error 404:

Creamos el (código Fuente: P4/S1/ejercicio3_get_tarea_error.py) para solucionar el cuerpo del mensaje HTML añadimos el método `@app.errorhandler(404)` que utiliza una función `make_response()` para hacer que en vez de devolver el HTML devuelva un objeto JSON.

Volvemos a solicitar una tarea, ahora obtenemos en el JSON “tarea inexistente”.

```
(flaskenv) uo301402@sd-uo301402:~$ nano ejercicio3_get_tarea_error.py
(flaskenv) uo301402@sd-uo301402:~$ http GET http://localhost:5000/lista/v1/tarea/7
HTTP/1.1 404 NOT FOUND
Connection: close
Content-Length: 30
Content-Type: application/json
Date: Mon, 17 Nov 2025 16:15:44 GMT
Server: gunicorn

{
  "error": "Tarea inexistente"
}
```

3.3: POST /lista/v1/tareas

Experimento 3.

(Código Fuente: P4/S1/experimento3.py)

Modificamos el `create_tarea()` para imprimir en la consola del servidor el contenido de la petición (`request.data`).

Ahora las peticiones http: Si no se envían datos, `request.data` está vacío. Si se envían datos (descripción=Prueba y completada:=false), llegan codificados en bytes.

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http POST http
://localhost:4000/lista/v1/tareas descripcion=Prueba
HTTP/1.1 201 CREATED
Connection: close
Content-Length: 2
Content-Type: text/html; charset=utf-8
Date: Mon, 17 Nov 2025 16:25:47 GMT
Server: gunicorn

OK
```

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http POST http
://localhost:4000/lista/v1/tareas
HTTP/1.1 201 CREATED
Connection: close
Content-Length: 2
Content-Type: text/html; charset=utf-8
Date: Mon, 17 Nov 2025 16:24:25 GMT
Server: gunicorn

OK
```

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http POST http
://localhost:4000/lista/v1/tareas completada:=false
HTTP/1.1 201 CREATED
Connection: close
Content-Length: 2
Content-Type: text/html; charset=utf-8
Date: Mon, 17 Nov 2025 16:26:51 GMT
Server: gunicorn

OK
```

Experimento 4.

Modificamos el código para que en lugar de imprimir request.data imprima request.json. Realizamos las peticiones http POST para probar esto de varias formas:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http POST http://localhost:5001/lista/v1/tareas
HTTP/1.1 415 UNSUPPORTED MEDIA TYPE
Connection: close
Content-Length: 215
Content-Type: text/html; charset=utf-8
Date: Mon, 17 Nov 2025 16:50:30 GMT
Server: gunicorn

<!doctype html>
<html lang=en>
<title>415 Unsupported Media Type</title>
<h1>Unsupported Media Type</h1>
<p>Did not attempt to load JSON data because the request Content-Type was not &#39;application/json&#39;.</p>

(flaskenv) uo301402@sd-uo301402:~/P4$ http POST http://localhost:5001/lista/v1/tareas d
descripcion=Prueba
HTTP/1.1 201 CREATED
Connection: close
Content-Length: 2
Content-Type: text/html; charset=utf-8
Date: Mon, 17 Nov 2025 16:53:04 GMT
Server: gunicorn

OK

(flaskenv) uo301402@sd-uo301402:~/P4$ http POST http://localhost:5001/lista/v1/tareas c
completadas=false
HTTP/1.1 201 CREATED
Connection: close
Content-Length: 2
Content-Type: text/html; charset=utf-8
Date: Mon, 17 Nov 2025 16:54:36 GMT
Server: gunicorn

OK
```

En la consola del servidor (Gunicorn), se observa que Flask ha convertido el JSON recibido en un diccionario nativo de Python. Si realizamos una petición sin cuerpo o con un JSON vacío, request.json será None.

```
(flaskenv) uo301402@sd-uo301402:~/P4$ gunicorn -b "0.0.0.0:5001" -w 1 experimento4:app
[2025-11-17 16:56:49 +0000] [1918] [INFO] Starting gunicorn 23.0.0
[2025-11-17 16:56:49 +0000] [1918] [INFO] Listening at: http://0.0.0.0:5001 (1918)
[2025-11-17 16:56:49 +0000] [1918] [INFO] Using worker: sync
[2025-11-17 16:56:49 +0000] [1919] [INFO] Booting worker with pid: 1919
Recibido POST con datos:
{'descripcion': 'Prueba'}
-----
Recibido POST con datos:
{'completadas': False}
-----
[]
```

Ejercicio 4 Agregamos POST tareas:

(Código fuente: P4/S1/ejercicio4_post_tareas.py)

Implementamos la ruta POST para crear las tareas y las almacenamos en memoria.

Desarrollamos la función create_tarea():

- Comprobamos si la petición contiene un cuerpo JSON válido y si el campo obligatorio descripción está sino lanzamos el error 400.
- Verificamos si el campo 'completada' es booleano sino 'False' por defecto.
- ID de la nueva tarea será 'tareas[-1]['id'] + 1'
- Creamos un errorhandler(400) para asegurar que, si falla, el cliente reciba un JSON
- El cliente es informado del ID con un JSON y el código 201

Verificamos el funcionamiento con una petición POST para crear una nueva tarea con éxito y una tarea con un error('completada:=false' ya está por defecto) para comprobar si funciona el código 201 y el 400:

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http POST http://localhost:3000/lista/v1/tareas descripcion="Otra tarea" completada=true
HTTP/1.1 201 CREATED
Connection: close
Content-Length: 64
Content-Type: application/json
Date: Mon, 17 Nov 2025 17:00:42 GMT
Server: gunicorn

{
  "tarea": {
    "completada": true,
    "descripcion": "Otra tarea",
    "id": 3
  }
}

(flaskenv) uo300480@sd-300480:~/TO-DO$ http POST http://localhost:3000/lista/v1/tareas completada:=false
HTTP/1.1 400 BAD REQUEST
Connection: close
Content-Length: 33
Content-Type: application/json
Date: Mon, 17 Nov 2025 17:01:23 GMT
Server: gunicorn

{
  "error": "Solicitud incorrecta"
}
```

3.4: PUT /lista/v1/tarea/<id>

Ejercicio 5: PUT tareas para actualizarlas:

Implementamos un método “update_tarea(id)” para la ruta PUT. El método consiste en buscar una tarea para luego actualizarla. Primero verifica si el cliente envía los campos “descripción” o “completada”, después verifica si el tipo de estos es el correcto. Si todo está bien, la tarea se actualiza.

Añadimos la tarea “Comprar el pan” no completada:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http PUT http://localhost:5001/lista/v1/tarea/1 d
descripcion="Comprar el pan" completada=false
HTTP/1.1 200 OK
Connection: close
Content-Length: 69
Content-Type: application/json
Date: Mon, 17 Nov 2025 17:31:21 GMT
Server: gunicorn

{
  "tarea": {
    "completada": false,
    "descripcion": "Comprar el pan",
    "id": 1
  }
}
```

Comprobamos que da error si introducimos un string en un campo de tipo booleano:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http PUT http://localhost:5001/lista/v1/tarea/1 c
completada="comprar el pan"
HTTP/1.1 400 BAD REQUEST
Connection: close
Content-Length: 33
Content-Type: application/json
Date: Mon, 17 Nov 2025 17:44:52 GMT
Server: gunicorn

{
  "error": "Solicitud incorrecta"
}
```

3.5.5 DELETE /lista/v1/tarea/<id>

Ejercicio 6 DELETE para borrar tareas:

Implementamos un método para eliminar la tarea indicada por el usuario (Código fuente: P4/S1/ejercicio6_delete_tarea.py)

Experimento 5:

Lanzamos de nuevo el servidor gunicorn, pero con 4 workers:

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ gunicorn -b "0.0.0.0:5000" -w 4 ejercicio6_delete_tarea:app
[2025-11-18 14:06:13 +0000] [1649] [INFO] Starting gunicorn 23.0.0
[2025-11-18 14:06:13 +0000] [1649] [INFO] Listening at: http://0.0.0.0:5000 (1649)
[2025-11-18 14:06:13 +0000] [1649] [INFO] Using worker: sync
[2025-11-18 14:06:14 +0000] [1650] [INFO] Booting worker with pid: 1650
[2025-11-18 14:06:14 +0000] [1651] [INFO] Booting worker with pid: 1651
[2025-11-18 14:06:14 +0000] [1652] [INFO] Booting worker with pid: 1652
[2025-11-18 14:06:14 +0000] [1654] [INFO] Booting worker with pid: 1654
```

Creamos una nueva tarea con POST:

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http POST http://localhost:5000/lista/v1/tareas descripcion="TareaA"
HTTP/1.1 201 CREATED
Connection: close
Content-Length: 61
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:07:47 GMT
Server: gunicorn

{
  "tarea": {
    "completada": false,
    "descripcion": "TareaA",
    "id": 3
  }
}
```

Al hacer la petición GET tareas, a veces sale la nueva tarea y otras no porque nos atiende un worker distinto. A la tercera petición nos mostró la tarea nueva, pero ya en la siguiente dejó de mostrarla. Non funcionó porque lo atendió el Worker A y otras veces no la vemos cuando atienden los otros. Como se puede ver en las imágenes:

<pre>(flaskenv) uo300480@sd-300480:~/TO-DO\$ http GET http://localhost:5000/lista/v1/tareas HTTP/1.1 200 OK Connection: close Content-Length: 240 Content-Type: application/json Date: Tue, 18 Nov 2025 14:09:55 GMT Server: gunicorn { "tareas": [{ "completada": true, "descripcion": "Terminar práctica Hola Mundo con Flask", "id": 1 }, { "completada": false, "descripcion": "Terminar práctica aplicación To-Do", "id": 2 }, { "completada": false, "descripcion": "TareaA", "id": 3 }] }</pre>	<pre>(flaskenv) uo300480@sd-300480:~/TO-DO\$ http GET http://localhost:5000/lista/v1/tareas HTTP/1.1 200 OK Connection: close Content-Length: 189 Content-Type: application/json Date: Tue, 18 Nov 2025 14:11:09 GMT Server: gunicorn { "tareas": [{ "completada": true, "descripcion": "Terminar práctica Hola Mundo con Flask", "id": 1 }, { "completada": false, "descripcion": "Terminar práctica aplicación To-Do", "id": 2 }] }</pre>
---	--

Probamos a borrar la tarea 1 con DELETE:

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http DELETE http://localhost:5000/lista/v1/tarea/1
HTTP/1.1 200 OK
Connection: close
Content-Length: 17
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:14:03 GMT
Server: gunicorn

{
  "borrado": true
}
```

A veces la tarea 1 desaparece y otras no, eso depende del worker que la procese.

Aquí ya la procesó este worker.

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http GET http://localhost:5000/lista/v1/tareas
HTTP/1.1 200 OK
Connection: close
Content-Length: 153
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:16:18 GMT
Server: gunicorn

{
  "tareas": [
    {
      "completada": false,
      "descripcion": "Terminar práctica aplicación To-Do",
      "id": 2
    },
    {
      "completada": false,
      "descripcion": "TareaA",
      "id": 3
    }
  ]
}
```

Vamos a modificar la tarea id=3 con PUT a ver que worker la procesa.

Lo que ocurre es que en la primera petición la procesa un worker que no contiene la tarea con id=3. En la siguiente petición encontramos al worker que sí que la tiene y la modificamos.

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http PUT http://localhost:5000/lista/v1/tarea/3 descripcion="Tarea finalizada" completada=true
HTTP/1.1 404 NOT FOUND
Connection: close
Content-Length: 30
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:23:22 GMT
Server: gunicorn

{
  "error": "Tarea inexistente"
}

(flaskenv) uo300480@sd-300480:~/TO-DO$ http PUT http://localhost:5000/lista/v1/tarea/3 descripcion="Tarea finalizada" completada=true
HTTP/1.1 200 OK
Connection: close
Content-Length: 70
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:23:27 GMT
Server: gunicorn

{
  "tarea": {
    "completada": true,
    "descripcion": "Tarea finalizada",
    "id": 3
  }
}
```

Aquí se puede ver la petición GET del worker que si procesó la tarea:

```
(flaskenv) uo300480@sd-300480:~/TO-DO$ http GET http://localhost:5000/lista/v1/tareas
HTTP/1.1 200 OK
Connection: close
Content-Length: 162
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:24:51 GMT
Server: gunicorn

{
  "tarefas": [
    {
      "completada": false,
      "descripcion": "Terminar práctica aplicación To-Do",
      "id": 2
    },
    {
      "completada": true,
      "descripcion": "Tarea finalizada",
      "id": 3
    }
  ]
}
```

Al usar -w 4, gunicorn crea 4 procesos independientes. Cada proceso tiene su propia copia de la memoria y sus propias variables Python.

Si el Worker 1 añade una tarea a su lista, el Worker 2 no se entera. Para arreglarlo, haría falta una base de datos para que los workers funcionen, puedan acceder a ella, se almacenen las tareas en común.

3.6: Nivel 3: HATEOAS

Ejercicio 7 Hateoas:

El objetivo es que, en el JSON retornado, nos muestre la url en lugar del número de la tarea. Esto nos beneficia. Por ejemplo, el cliente no se tendría que saber la url en caso de un cambio en la estructura. Esto lo conseguimos con la función proporcionada por flask, url_for(). Lo que hace es obtener la ruta de la función pasada como parámetro.

Comprobamos si devuelve la url en cada una de las peticiones:

Petición GET:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http GET http://localhost:5001/lista/v1/tareas
HTTP/1.1 200 OK
Connection: close
Content-Length: 269
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:45:57 GMT
Server: gunicorn

{
  "tarefas": [
    {
      "completada": true,
      "descripcion": "Terminar práctica Hola Mundo con Flask",
      "uri": "http://localhost:5001/lista/v1/tarea/1"
    },
    {
      "completada": false,
      "descripcion": "Terminar práctica aplicación To-Do",
      "uri": "http://localhost:5001/lista/v1/tarea/2"
    }
  ]
}
```

Petición POST:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http POST http://localhost:5001/lista/v1/tareas d
descripcion="Terminar practica SD"
HTTP/1.1 201 CREATED
Connection: close
Content-Length: 115
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:48:14 GMT
Server: gunicorn

{
  "tarea": {
    "completada": false,
    "descripcion": "Terminar practica SD",
    "uri": "http://localhost:5001/lista/v1/tarea/3"
  }
}
```

Petición PUT:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http PUT http://localhost:5001/lista/v1/tarea/1 d
descripcion="Comprar el pan" completada:=false
HTTP/1.1 200 OK
Connection: close
Content-Length: 109
Content-Type: application/json
Date: Tue, 18 Nov 2025 14:52:35 GMT
Server: gunicorn

{
  "tarea": {
    "completada": false,
    "descripcion": "Comprar el pan",
    "uri": "http://localhost:5001/lista/v1/tarea/1"
  }
}
```

4. (Parte 3) Clientes para la aplicación To-Do.

En esta parte 3, el objetivo fue desarrollar interfaces de usuario (clientes) que consuman la API RESTful que hemos implementado. Trabajamos con una interfaz dada y mediante la línea de comandos.

4.1: Cliente mediante interfaz gráfica

Utilizamos el código main.html para tener una interfaz web.

(Código fuente: P4/S2-Cliente/static)

Ejercicio 1 Modificación hateos:

Esto es lo que ocurre cuando eliminamos la lista de tareas e intentamos añadir una nueva. Corregimos añadiendo una condición, si la lista está vacía id=1. Modificamos en P4/S2-Cliente/ejercicio7_hateos.py esa condición.

```
[2025-11-19 22:31:24,845] ERROR in app: Exception on /lista/v1/tareas [POST]
Traceback (most recent call last):
  File "/home/uo300480/flaskenv/lib/python3.12/site-packages/flask/app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
               ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/uo300480/flaskenv/lib/python3.12/site-packages/flask/app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/uo300480/flaskenv/lib/python3.12/site-packages/flask/app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
         ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/uo300480/flaskenv/lib/python3.12/site-packages/flask/app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)  # type: ignore[no-any-return]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/uo300480/TO-DO/S2-Cliente/ejercicio7_hateos.py", line 59, in create_tarea
    'id': tareas[-1]['id'] + 1,
              ~~~~~^
IndexError: list index out of range
```

4.2: Políticas CORS

Al intentar conectar este cliente web con otro servidor en otro origen, el servidor de un compañero. El navegador bloquea las peticiones por seguridad. Para solucionarlo implementamos el `@app.after_request`.

Ejercicio 2 Política CORS:

Para comprobar que ahora funciona esto. Desde mi ordenador lanzo el servidor con IP:192.168.207.139 para que mi compañero pruebe la conexión.

```
(flaskenv) uo300480@sd-02:~/P4/S2-Cliente$ gunicorn -b "192.168.207.139:5600" -w 1 ejercicio7_hateos:app
[2025-11-20 15:50:01 +0000] [3000] [INFO] Starting gunicorn 23.0.0
[2025-11-20 15:50:01 +0000] [3000] [INFO] Listening at: http://192.168.207.139:5600 (3000)
[2025-11-20 15:50:01 +0000] [3000] [INFO] Using worker: sync
[2025-11-20 15:50:01 +0000] [3001] [INFO] Booting worker with pid: 3001
```

Esta es la interfaz de la lista de tareas inicial desde el ordenador.

Prueba de la petición GET desde otro ordenador, utilizando la IP:192.168.207.139 y vemos que hay conexión y que obtiene la misma lista de tareas.

Lista de tareas

Pendientes 2 de 4 ([Eliminar](#) tareas completadas)

☒	Terminar práctica Hola-Mundo con Flask	X
☐	Terminar práctica aplicación To-Do	X
☒	Nueva	X
☐	gjkrgd	X

Servidor por defecto (el que sirvió el html)

Cargar

Pulsa "Cargar" para recargar la lista de tareas o cambiar el servidor.

```
(flaskenv) wo301402@sd-01:~$ http GET http://192.168.207.139:5600/lista/v1/tareas
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: close
Content-Length: 473
Content-Type: application/json
Date: Thu, 20 Nov 2025 15:50:35 GMT
Server: gunicorn

{
  "tareas": [
    {
      "completada": true,
      "descripcion": "Terminar práctica Hola Mundo con Flask",
      "uri": "http://192.168.207.139:5600/lista/v1/tarea/1"
    },
    {
      "completada": false,
      "descripcion": "Terminar práctica aplicación To-Do",
      "uri": "http://192.168.207.139:5600/lista/v1/tarea/2"
    },
    {
      "completada": true,
      "descripcion": "Nueva",
      "uri": "http://192.168.207.139:5600/lista/v1/tarea/3"
    },
    {
      "completada": false,
      "descripcion": "gjkrgd",
      "uri": "http://192.168.207.139:5600/lista/v1/tarea/4"
    }
  ]
}
```

Ahora probamos a crear una nueva tarea utilizando la petición POST <IP>, y vemos la interfaz del cliente desde otro ordenador:

Lista de tareas

Pendientes 2 de 4 ([Eliminar](#) tareas completadas)

☒	Terminar práctica Hola-Mundo con Flask	X
☐	Terminar práctica aplicación To-Do	X
☒	Nueva-tarea	X
☐	mesi	X

Servidor por defecto (el que sirvió el html)

Cargar

Pulsa "Cargar" para recargar la lista de tareas o cambiar el servidor.

```
(flaskenv) wo301402@sd-01:~$ http POST http://192.168.207.139:5700/lista/v1/tareas descripcion="mesi"
HTTP/1.1 201 CREATED
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: close
Content-Length: 105
Content-Type: application/json
Date: Thu, 20 Nov 2025 15:54:20 GMT
Server: gunicorn

{
  "tarea": {
    "completada": false,
    "descripcion": "mesi",
    "uri": "http://192.168.207.139:5700/lista/v1/tarea/4"
  }
}
```

4.3. Cliente mediante línea de comandos

Ejercicio 3 TO-DO cliente click:

(Código fuente: P4/S2-Cliente/ToDo_cliente_click.py)

Para hacer este cliente mediante línea de comandos vamos a utilizar la librería “click”. Implementamos los comandos (uncheck, delete y edit). Cada comando tiene su función, el comando “uncheck” marca una tarea como pendiente, “delete” borra una tarea, y “edit” modifica la descripción de una tarea.

Comprobamos que los tres comandos base funcionan correctamente (add, list, check).

```
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py add "Hacer P5" False
Nueva tarea añadida:
URI: http://localhost:5001/lista/v1/tarea/3 - Hacer P5 - Pendiente
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py list
URI: http://localhost:5001/lista/v1/tarea/1 - Terminar práctica Hola Mundo con Flask - Completada
URI: http://localhost:5001/lista/v1/tarea/2 - Terminar práctica aplicación To-Do - Pendiente
URI: http://localhost:5001/lista/v1/tarea/3 - Hacer P5 - Pendiente
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py check 3
Tarea 3 marcada como completada.
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py list
URI: http://localhost:5001/lista/v1/tarea/1 - Terminar práctica Hola Mundo con Flask - Completada
URI: http://localhost:5001/lista/v1/tarea/2 - Terminar práctica aplicación To-Do - Pendiente
URI: http://localhost:5001/lista/v1/tarea/3 - Hacer P5 - Completada
```

La implementación del comando “uncheck” es la misma que “check” cambiando el estado de completada a False. Para el comando “delete” usamos el método. delete() y verificamos si el borrado ha sido exitoso y si la tarea existe. Por último, en el comando “edit” usamos el método. put() y verificamos con el código de respuesta si la descripción ha sido actualizada y si la tarea existe.

Comprobamos el funcionamiento de los tres comandos restantes (uncheck, delete, edit).

```
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py list
URI: http://localhost:5001/lista/v1/tarea/1 - Terminar práctica Hola Mundo con Flask - Completada
URI: http://localhost:5001/lista/v1/tarea/2 - Terminar práctica aplicación To-Do - Pendiente
URI: http://localhost:5001/lista/v1/tarea/3 - Hacer P5 - Completada
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py uncheck 1
Tarea 1 marcada como pendiente.
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py delete 2
Tarea 2 eliminada correctamente.
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py edit 3 "Hacer P5 y terminar P4"
Descripción de la tarea 3 actualizada a: 'Hacer P5 y terminar P4'
(flaskenv) uo301402@sd-uo301402:~/P4$ python3 ToDo_cliente_click.py list
URI: http://localhost:5001/lista/v1/tarea/1 - Terminar práctica Hola Mundo con Flask - Pendiente
URI: http://localhost:5001/lista/v1/tarea/3 - Hacer P5 y terminar P4 - Completada
```

5. (Parte 4): Aplicación To-Do (BBDD y Autenticación)

En esta parte 4, Creamos la base de datos para la aplicación y el sistema de Autenticación.

5.1 Base de Datos

Creamos una base de datos MariaDB en la que podemos conectarnos de esta manera:

```
sudo mysql -u root -p Nos pedirá una contraseña: root_pass
```

```
Nos conectamos a la base de datos CONNECT tarea_db;
```

Ejercicio 1 Implementamos la Base de Datos:

En este ejercicio sustituimos los métodos del ejercicio anterior para implementar la base de datos, Definimos la conexión a la BBDD, y adaptamos los métodos:

GET (Lista): Sustituimos el acceso a la lista por `Tarea.query.all()`.

GET (Elemento): Utilizamos `Tarea.query.get(id)` para buscar por clave

POST: Objeto Tarea y lo guardamos con `db.session.add()` y `db.session.commit()`. Y el ID lo genera la base de datos.

PUT: Recuperamos el objeto con `query.get()`, modificamos sus atributos y confirmamos con `commit()`.

DELETE: Usamos `db.session.delete()` seguido de `commit()`.

(Código fuente: P4/S4-BBDD/ejercicio1_bbddToDo.py)

Probamos lanzando el servidor con la nueva implementación:

Lanzamos servidor:

```
gunicorn -w 1 -b 0.0.0.0:5000 ejercicio1_bbddToDo:app
```

Creamos una tarea desde la interfaz, y vamos a la base de datos para ver si está ahí.

Como se puede ver funciona bien la base de datos:

Lista de tareas

Pendientes 2 de 3 (Eliminar tareas completadas)

☒	Tarea 1	X
☐	Tarea 2	X
☐	Probar base de datos	X

+ Escribe aquí una tarea nueva

Servidor por defecto (el que sirvió el html)

Pulsa "Cargar" para recargar la lista de tareas o cambiar el servidor.

```
MariaDB [tarea_db]> SELECT * FROM tareas;
+-----+-----+-----+
| id | descripcion | completada |
+-----+-----+-----+
| 1 | Tarea 1 | 1 |
| 2 | Tarea 2 | 0 |
+-----+-----+-----+
2 rows in set (0,001 sec)

MariaDB [tarea_db]> SELECT * FROM tareas;
+-----+-----+-----+
| id | descripcion | completada |
+-----+-----+-----+
| 1 | Tarea 1 | 1 |
| 2 | Tarea 2 | 0 |
| 3 | Probar base de datos | 0 |
+-----+-----+-----+
3 rows in set (0,003 sec)

MariaDB [tarea_db]> SHOW COLUMNS FROM tareas;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int(11) | NO | PRI | NULL | auto_increment |
| descripcion | varchar(100) | NO | | NULL | |
| completada | tinyint(1) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

Vamos a probar a desconectar el servidor, de momento en la aplicación de tareas tenemos:

Lista de tareas

Pendientes 1 de 2 (Eliminar tareas completadas)

☒	Probar base de datos	X
☐	Desconectar servidor	X

+ Escribe aquí una tarea nueva

Servidor por defecto (el que sirvió el html)

Pulsa "Cargar" para recargar la lista de tareas o cambiar el servidor.

Lo detenemos:

```
(flaskenv) uo300480@sd-300480:~/TO-DO/S3-BBDD$ gunicorn -w 1 -b 0.0.0.0:5000 ejercicio1_bbddToDo:app
[2025-11-25 13:17:10 +0000] [2337] [INFO] Starting gunicorn 23.0.0
[2025-11-25 13:17:10 +0000] [2337] [INFO] Listening at: http://0.0.0.0:5000 (2337)
[2025-11-25 13:17:10 +0000] [2337] [INFO] Using worker: sync
[2025-11-25 13:17:10 +0000] [2338] [INFO] Booting worker with pid: 2338
^C[2025-11-25 13:22:36 +0000] [2337] [INFO] Handling signal: int
[2025-11-25 13:22:36 +0000] [2338] [INFO] Worker exiting (pid: 2338)
[2025-11-25 13:22:36 +0000] [2337] [INFO] Shutting down: Master
(flaskenv) uo300480@sd-300480:~/TO-DO/S3-BBDD$
```

Volvemos a conectar:

```
(flaskenv) uo300480@sd-300480:~/TO-DO/S3-BBDD$ gunicorn -w 1 -b 0.0.0.0:5000 ejercicio1_bbddToDo:app
[2025-11-25 13:23:14 +0000] [2357] [INFO] Starting gunicorn 23.0.0
[2025-11-25 13:23:14 +0000] [2357] [INFO] Listening at: http://0.0.0.0:5000 (2357)
[2025-11-25 13:23:14 +0000] [2357] [INFO] Using worker: sync
[2025-11-25 13:23:14 +0000] [2358] [INFO] Booting worker with pid: 2358
```

Vemos que sigue funcionando y muestra las tareas que teníamos:

Lista de tareas

Pendientes 1 de 2 (Eliminar tareas completadas)

- ☒ Probar base de datos
- ☐ Desconectar servidor

+ Escribe aquí una tarea nueva

Servidor por defecto (el que sirvió el html)

http://p.puerto

Pulsa "Cargar" para recargar la lista de tareas o cambiar el servidor.

```
MariaDB [tarea_db]> SELECT * FROM tareas;
+-----+-----+-----+
| id | descripcion          | completada |
+-----+-----+-----+
| 3 | Probar base de datos |          1 |
| 4 | Desconectar servidor |          0 |
+-----+-----+-----+
2 rows in set (0,002 sec)
```

Ejercicio 2: Probamos con 4 workers.

Vamos a probar que pasa ahora con 4 workers. Lanzamos el servidor de nuevo:

```
(flaskenv) wo300480@sd-300480:~/TO-DO/S3-BBDD$ gunicorn -w 4 -b 0.0.0.0:5000 ejercicio1_bbddToDo:app
[2025-11-25 13:28:31 +0000] [2368] [INFO] Starting gunicorn 23.0.0
[2025-11-25 13:28:31 +0000] [2368] [INFO] Listening at: http://0.0.0.0:5000 (2368)
[2025-11-25 13:28:31 +0000] [2368] [INFO] Using worker: sync
[2025-11-25 13:28:31 +0000] [2369] [INFO] Booting worker with pid: 2369
[2025-11-25 13:28:31 +0000] [2370] [INFO] Booting worker with pid: 2370
[2025-11-25 13:28:32 +0000] [2371] [INFO] Booting worker with pid: 2371
[2025-11-25 13:28:32 +0000] [2373] [INFO] Booting worker with pid: 2373
```

Probamos a eliminar y agregar una tarea nueva y vemos que en la base de datos siguen saliendo bien.

Lista de tareas

Pendientes 2 de 2 (Eliminar tareas completadas)

- ☐ Desconectar servidor
- ☐ Nueva tarea

+ Escribe aquí una tarea nueva

Servidor por defecto (el que sirvió el html)

http://p.puerto

Pulsa "Cargar" para recargar la lista de tareas o cambiar el servidor.

```
MariaDB [tarea_db]> SELECT * FROM tareas;
+-----+-----+-----+
| id | descripcion          | completada |
+-----+-----+-----+
| 4 | Desconectar servidor |          0 |
| 5 | Nueva tarea          |          0 |
+-----+-----+-----+
2 rows in set (0,002 sec)
```

En la tarea anterior si lanzábamos 4 workers, gunicorn crea 4 procesos independientes con su propia copia de la memoria y sus propias variables Python. Y dependiendo de que worker atendía la petición en unos aparecía como “desaparecida”

Añadiendo la BBDD el almacenamiento ahora se hace mediante la base de datos MariaDB, ahora los 4 workers sin estado, se conectan a la base de datos y pueden atender las tareas q hay almacenadas.

5.2 Seguridad basada en Autenticación

Incorporaremos un mecanismo de control de acceso basado en HTTP Basic Auth, para asegurar quien puede acceder y utilizamos la librería Werkzeug para generar un hash seguro de la contraseña alumno_pass.

5.2.2 Implementación de Seguridad Básica (HTTP Auth) y Hash_

Implementamos un mecanismo de autenticación utilizando el estándar HTTP Basic Auth, para ello integramos la librería Flask-HTTPAuth.

Ejercicio 3 Autenticación:

(Código fuente: P4/S4-BBDD/ejercicio3_authToDo.py)

Añadimos la línea (auth.realm = "Es necesario autenticarse") que se enviará al cliente cuando se requiera autenticación. Se mostrará en la cabecera WWW-Authenticate de la respuesta HTTP.

Probamos a enviar la cabecera con --auth nombre:clave en el método GET.

Vamos a probar con una verificación fallida, ponemos la contraseña mal y obtenemos un error 401:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http --auth alumno:ep1f GET localhost:5001/lista/v1/tareas
HTTP/1.1 401 UNAUTHORIZED
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: close
Content-Length: 41
Content-Type: application/json
Date: Tue, 25 Nov 2025 17:22:25 GMT
Server: gunicorn
WWW-Authenticate: Basic realm="Es necesario autenticarse"

{
  "error": "Credenciales no válidas"
}
```

Ponemos la contraseña bien y obtenemos la verificación correcta:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http --auth alumno:alumno_pass GET localhost:5001/lista/v1/tareas
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: close
Content-Length: 14
Content-Type: application/json
Date: Tue, 25 Nov 2025 17:15:06 GMT
Server: gunicorn

{
  "tareas": []
}
```

Ejercicio 4: Validación del Cliente Web con Autenticación

Primero, introducimos unas credenciales incorrectas:



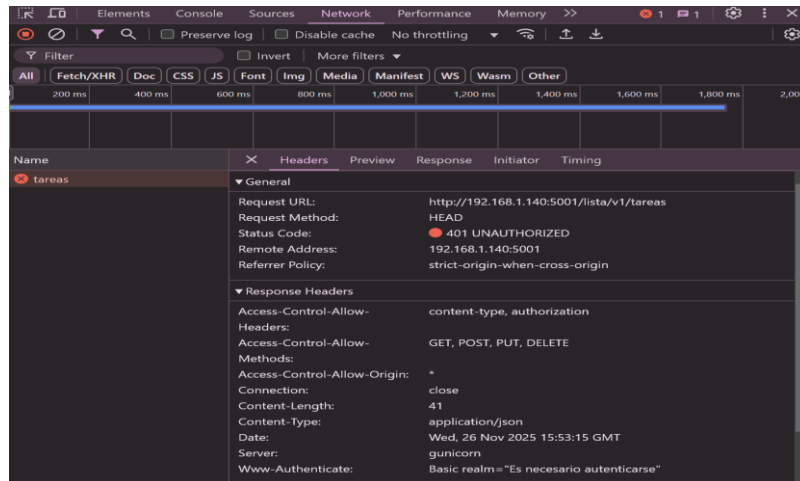
Servidor Cargar

Deja en blanco para usar como servidor REST el mismo que sirvió esta página.
Escribe `http://ip:puerto` para conectar con otro servidor via REST

Usuario

Contraseña

Las credenciales son incorrectas



Ahora introducimos las credenciales correctas y modificamos la lista de tareas.



Lista de tareas

Pendientes 1 de 1 ([Eliminar](#) tareas completadas)

☐ Comprar el pan

Servidor Cargar

Deja en blanco para usar como servidor REST el mismo que sirvió esta página.
Escribe `http://ip:puerto` para conectar con otro servidor via REST

```
MariaDB [tarea_db]> SELECT * FROM tareas;
+----+-----+-----+
| id | descripcion | completada |
+----+-----+-----+
| 1  | Comprar el pan | 0          |
+----+-----+-----+
1 row in set (0,002 sec)
```

Ejercicio 5: Fortalecimiento de la seguridad mediante Hashing.

El objetivo de este ejercicio es eliminar el almacenamiento de contraseñas en texto. Para conseguir esto vamos a almacenar la contraseña en un fichero .txt, y a usar el hash en lugar de texto plano. Para obtener el hash de la contraseña utilizaremos la función “generate_password_hash()” de Werkzeug. Modificamos también la función verificar() para que compruebe si el usuario está en el diccionario “autorizados”, con la función check_password_hash().

Comprobamos con las credenciales correctas para verificar el funcionamiento:

```
(flaskenv) uo301402@sd-uo301402:~/P4$ http --auth alumno:alumno_pass GET localhost:5001/lista/v1/tareas
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: close
Content-Length: 112
Content-Type: application/json
Date: Wed, 26 Nov 2025 16:46:56 GMT
Server: gunicorn

{
  "tareas": [
    {
      "completada": false,
      "descripcion": "Comprar el pan",
      "uri": "http://localhost:5001/lista/v1/tarea/1"
    }
  ]
}
```

Lista de tareas

Pendientes 1 de 1 ([Eliminar](#) tareas completadas)

☐ Comprar el pan ✕

+

Servidor

Cargar

Deja en blanco para usar como servidor REST el mismo que sirvió esta página.

Escribe `http://ip:puerto` para conectar con otro servidor via REST

Vemos que sigue funcionando correctamente.

6. (Parte 5) Despliegue en Docker.

En esta parte 5 lanzamos contenedores para el despliegue completo de la aplicación de gestión To-Do, utilizando la red virtual llamada "pruebas". El despliegue final es una infraestructura de tres contenedores: la base de datos MariaDB, la aplicación Flask desplegada con gunicorn y un servidor web (nginx).

6.1 Contenedor MariaDB.

Lanzamos el contenedor oficial de MariaDB, mediante el comando:

```
$ docker run -d --name mariadb_tarea_db --rm -v $HOME/basedatos:/var/lib/mysql \
--network pruebas -e MYSQL_ROOT_PASSWORD=root_pass -e MYSQL_DATABASE=tarea_db \
-e MYSQL_USER=tarea_user -e MYSQL_PASSWORD=tarea_pass mariadb:latest
```

Para la conexión con la base de datos:

```
$ docker run --rm -it --network pruebas mysql:latest bash
```

6.2 Gunicorn + Flask.

Creamos el archivo Dockerfile con el código necesario. Modificamos el ejercicio anterior para incluir la conexión con el contenedor de la base de datos.

Ejercicio 1 Configuramos conexión con la BBDD:

(Código fuente. P4/S5/ejercicio1_finalToDo.py).

Aquí cambiamos SQLALCHEMY_DATABASE_URI para la conexión con el contenedor de la base de datos.

Gracias al servidor DNS de la red virtual podemos indicar mariadb_tarea_db como dirección en la URL en lugar de tener que usar una IP.

Al lanzar los contenedores con la opción --network pruebas y asignarles un nombre con la opción --name, Docker permite que los contenedores dentro de esa red se conecten entre sí utilizando sus nombres como si fueran dominios.

Ejercicio 2: Construimos la imagen de nuestra aplicación y lanzamos el servidor.

1. Construimos la imagen de la aplicación con Docker:

- `'docker build -t tarea_app'`

2. Lanzamos el contenedor de la aplicación conectándolo al Puerto 5000 y a la red 'pruebas':

- `'docker run --name tarea_app_flask --rm -d --network pruebas -p 5000:5000 tarea_app'`

- con `'--name'` le damos el nombre al contenedor.
- con `'--rm'` se borra automáticamente con lo que tengamos.
- con `'-d'` lanza el contenedor en modo daemon
- con `'--network'` conectamos con la red que creamos antes(pruebas)
- `'tarea_app'` es la imagen construida anteriormente con `docker build -t tarea_app`.

Tras lanzar la imagen mariadb y el servidor comprobamos que esté corriendo:

```
no300480@sd-300480:~/sd-pl2-g14/P4/S5$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
ce4f552147f4   tarea_app      "gunicorn -b 0.0.0.0..." About a minute ago Up             0.0.0.0:5000->5000/tcp   tarea_app_flask
996dc6d2a4d6   mariadb:latest "docker-entrypoint.s..." 16 minutes ago Up             3306/tcp                mariadb_tarea_db
```

Ya tenemos las imágenes construidas en Docker y el servidor lanzado, vamos a probar si funciona. Entramos en `http:// IP:5000/static/main-auth.html` y nos autenticamos como 'alumno' y contraseñas: 'alumno_pass'.

No seguro 192.168.0.20:5000/static/main-auth.html Resu

Lista de tareas

Pendientes 0 de 0 (Eliminar tareas completadas)

+

Servidor Cargar

Deja en blanco para usar como servidor REST el mismo que sirvió esta página.

Escribe `http://ip:puerto` para conectar con otro servidor via REST

Creamos dos tareas y vamos a consultar la base de datos para ver si hay conexión

Lista de tareas

Pendientes 2 de 2 (Eliminar tareas completadas)

☐ una tarea

☐ otra tarea

+ Escribe aquí una tarea nueva

Servidor Cargar

Deja en blanco para usar como servidor REST el mismo que sirvió esta página.
Escribe <http://ip:puerto> para conectar con otro servidor via REST

```
mysql> SHOW TABLES;
+-----+
| Tables_in_tarea_db |
+-----+
| tareas              |
+-----+
1 row in set (0.003 sec)

mysql> SELECT * FROM tareas;
+-----+-----+-----+
| id | descripcion | completada |
+-----+-----+-----+
| 1  | una tarea   | 0          |
| 2  | otra tarea  | 0          |
+-----+-----+-----+
2 rows in set (0.005 sec)
```

Ejercicio 3: Comprobación persistencia.

Vamos a detener los contenedores -> 'docker stop tarea_app_flask mariadb_tarea_db'

No tenemos ningún contenedor corriendo.

Vamos a lanzar la base de datos mariadb y la tarea_app_flask.

```
uo300480@sd-300480:~/sd-pl2-g14/P4/S5$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
uo300480@sd-300480:~/sd-pl2-g14/P4/S5$ docker run -d --name mariadb_tarea_db --rm -v $HOME/basedatos:/var/lib/mysql \
--network pruebas -e MYSQL_ROOT_PASSWORD=root_pass -e MYSQL_DATABASE=tarea_db \
-e MYSQL_USER=tarea_user -e MYSQL_PASSWORD=tarea_pass mariadb:latest
efdf607c7dd8be5ae2bd70dff4da16351baa002cb64f3f8ea7876f8516a2ae6f
uo300480@sd-300480:~/sd-pl2-g14/P4/S5$ docker run --name tarea_app_flask --rm -d --network pruebas -p 5000:5000 tarea_
app
ef57a99645292fe0f3152f47aec84c5129ab583d8d425692ec63b8234570cddb
```

Nos conectamos a la base de datos y vemos que esta todo desplegado.

```
uo300480@sd-300480:~/sd-pl2-g14/P4/S5$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
a13b01d8ac9e   mysql:latest   "docker-entrypoint.s..." 56 seconds ago Up 55 seconds 3306/tcp, 33060/tcp
ef57a9964529   tarea_app      "gunicorn -b 0.0.0.0..." 3 minutes ago  Up 3 minutes  0.0.0.0:5000->5000/tcp, [::]
:5000->5000/tcp
efdf607c7dd8   mariadb:latest "docker-entrypoint.s..." 6 minutes ago  Up 6 minutes  3306/tcp
mariadb tarea db
```

Mostramos las tareas y vemos siguen apareciendo en la base de datos y en el cliente.
Probamos a agregar alguna tarea y a marcar como completada.

Lista de tareas

Pendientes 2 de 3 (Eliminar tareas completadas)

☒ una-tarea

☐ otra tarea

☐ nuevaaa

+ Escribe aquí una tarea nueva

Servidor Cargar

Deja en blanco para usar como servidor REST el mismo que sirvió esta página.
Escribe <http://ip:puerto> para conectar con otro servidor via REST

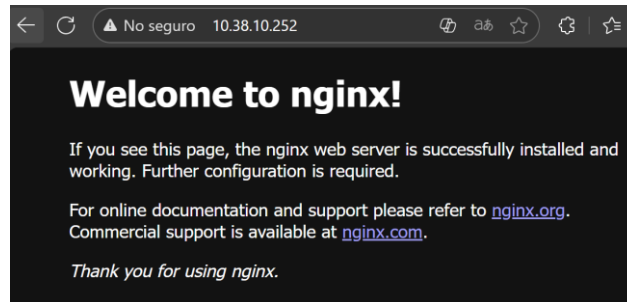
```
mysql> SELECT * FROM tareas;
+-----+-----+-----+
| id | descripcion | completada |
+-----+-----+-----+
| 1  | una tarea   | 0          |
| 2  | otra tarea  | 0          |
+-----+-----+-----+
2 rows in set (0.009 sec)

mysql> SELECT * FROM tareas;
+-----+-----+-----+
| id | descripcion | completada |
+-----+-----+-----+
| 1  | una tarea   | 1          |
| 2  | otra tarea  | 0          |
| 3  | nuevaaa    | 0          |
+-----+-----+-----+
3 rows in set (0.003 sec)
```

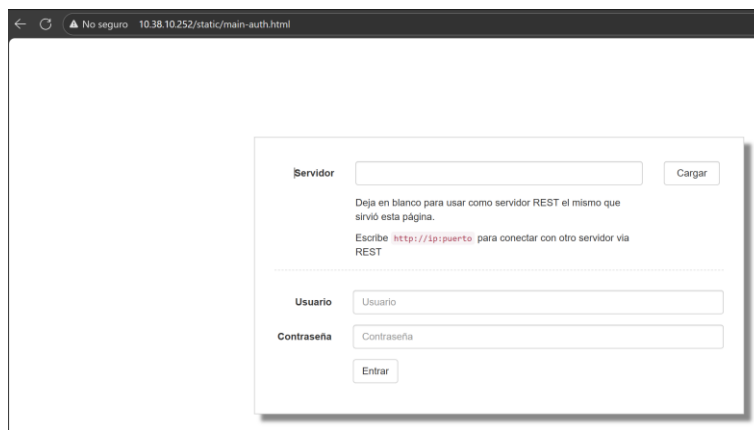
6.3 Contenedor nginx (servidor web)

Ejercicio 4: Probando Funcionamiento:

Comprobamos conexión con nginx en `http://10.38.10.252` vemos que funciona

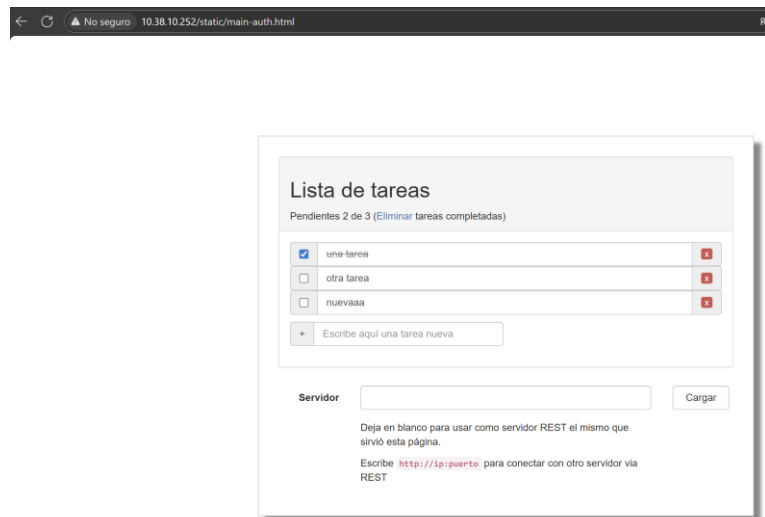


Comprobamos el funcionamiento de la interfaz cliente en `http://10.38.10.252/static/main-auth.html`:



Nos autenticamos con 'alumno' y 'alumno_pass', funciona correctamente.





Probamos conexión en `http://10.38.10.252/lista/v1/tareas` y nos autenticamos:



Obtenemos la lista de tareas correctamente:



- Desde la terminal:

GET: `http GET http://10.38.10.252/lista/v1/tareas --auth alumno:alumno_pass`

```
(flaskenv) wo300480@sd-300480:~/sd-pl2-g14/P4/S5$ http GET http://10.38.10.252/lista/v1/tareas --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 287
Content-Type: application/json
Date: Tue, 02 Dec 2025 18:23:37 GMT
Server: nginx/1.29.3

{
  "tareas": [
    {
      "completada": true,
      "descripcion": "una tarea",
      "uri": "http://10.38.10.252/lista/v1/tarea/1"
    },
    {
      "completada": false,
      "descripcion": "otra tarea",
      "uri": "http://10.38.10.252/lista/v1/tarea/2"
    },
    {
      "completada": false,
      "descripcion": "nuevaaa",
      "uri": "http://10.38.10.252/lista/v1/tarea/3"
    }
  ]
}
```

POST: http POST http://10.38.10.252/lista/v1/tareas descripción="desde POST" completada:=true --auth alumno:alumno_pass

```
(flaskenv) uo300480@sd-300480:~/sd-pl2-gl4/P4/S5$ http POST http://10.38.10.252/lista/v1/tareas descripcion="desde POST" completada:=true --auth alumno:alumno_pass
HTTP/1.1 201 CREATED
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 102
Content-Type: application/json
Date: Tue, 02 Dec 2025 18:26:02 GMT
Server: nginx/1.29.3

{
  "tarea": {
    "completada": true,
    "descripcion": "desde POST",
    "uri": "http://10.38.10.252/lista/v1/tarea/4"
  }
}
```

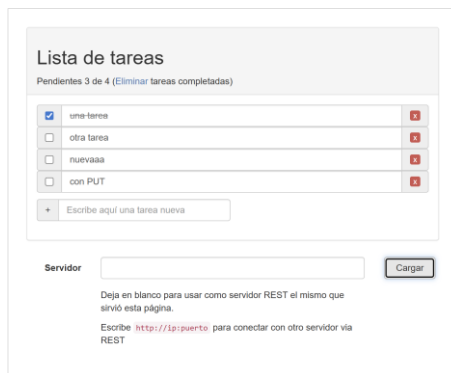
PUT: http PUT http://10.38.10.252/lista/v1/tarea/4 descripción="con PUT" completada:=false --auth alumno:alumno_pass

```
(flaskenv) uo300480@sd-300480:~/sd-pl2-gl4/P4/S5$ http PUT http://10.38.10.252/lista/v1/tarea/4 descripcion="con PUT" completada:=false --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 100
Content-Type: application/json
Date: Tue, 02 Dec 2025 18:27:20 GMT
Server: nginx/1.29.3

{
  "tarea": {
    "completada": false,
    "descripcion": "con PUT",
    "uri": "http://10.38.10.252/lista/v1/tarea/4"
  }
}
```

Se puede ver cómo funciona correctamente desde varios sitios:

Desde la línea de comandos, conectando con la base de datos, consultando las tareas y desde la interfaz web:



Lista de tareas
Pendientes 3 de 4 (Eliminar tareas completadas)

- ☒ una tarea
- ☐ otra tarea
- ☐ nuevaaa
- ☐ con PUT

Servidor:

Deja en blanco para usar como servidor REST el mismo que sirvió esta página.
Escribe <http://ip:puerto> para conectar con otro servidor via REST

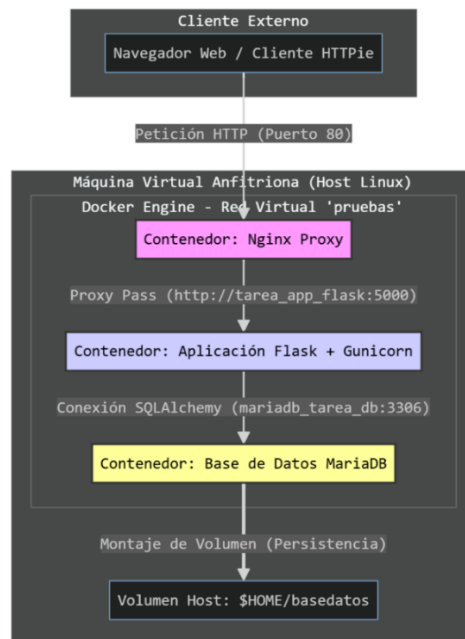
```
{
  "tareas": [
    {
      "completada": true,
      "descripcion": "una tarea",
      "uri": "http://10.38.10.252/lista/v1/tarea/1"
    },
    {
      "completada": false,
      "descripcion": "otra tarea",
      "uri": "http://10.38.10.252/lista/v1/tarea/2"
    },
    {
      "completada": false,
      "descripcion": "nuevaaa",
      "uri": "http://10.38.10.252/lista/v1/tarea/3"
    },
    {
      "completada": false,
      "descripcion": "con PUT",
      "uri": "http://10.38.10.252/lista/v1/tarea/4"
    }
  ]
}
```

```
mysql> CONNECT tarea_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Connection id: 8
Current database: tarea_db

mysql> SELECT * FROM tareas;
+----+-----+-----+
| id | descripcion | completada |
+----+-----+-----+
| 1 | una tarea | 1 |
| 2 | otra tarea | 0 |
| 3 | nuevaaa | 0 |
| 4 | con PUT | 0 |
+----+-----+-----+
4 rows in set (0.003 sec)
```

Ejercicio 5: Imagen Infraestructura:



Se puede observar:

Proxy Inverso (Nginx): Es el punto de entrada único desde el exterior. El puerto 80 de la máquina host se mapea al puerto 80 de este contenedor.

Backend API (Flask+Gunicorn): Este contenedor tiene la lógica de la aplicación. No es accesible directamente desde el exterior. Recibe las peticiones retransmitidas por Nginx a través del puerto 5000.

Base de Datos (MariaDB): Contenedor que garantiza la persistencia de los datos (tareas y usuarios).

6.4 Automatizar Despliegue.

Construimos un script que reinicie los contenedores. Primero, los detendrá. Después, los iniciará, dejando unos segundos de espera entre comandos, para dar tiempo a que se inicien los servicios de un contenedor antes de lanzar el siguiente.

7.CONCLUSIONES

Tras realizar esta práctica entera, hemos aprendido cómo evoluciona un servicio web desde inicio hasta la arquitectura final. Lo más importante que hemos aprendido ha sido la (ausencia de estado) en los servidores REST. Al principio, cuando guardábamos las tareas en una lista en memoria (Python), nos dimos cuenta de que al usar Gunicorn con varios workers, las tareas desaparecían o no eran accesibles por todos los procesos. Esto se arregla con una base de datos en sistemas distribuidos, para que los workers accedan a la misma información.

Finalmente, la parte de Docker ha sido la que más nos ha gustado para el despliegue. Es muy interesante ver cómo podemos levantar toda la infraestructura (Base de datos, API Flask y Proxy, Nginx) conectada mediante una red propia (pruebas). Esto facilita muchísimo llevar la aplicación a producción o a cualquier otra máquina sin problemas de librerías, etc.

8.PARTICIPACIÓN

Ambos contribuimos en todas las partes de esta práctica y en la creación de la memoria.

Adrián manso:

Parte 1: 50%

Parte 2: 50%

Parte 3: 50%

Parte 4: 50%

Parte 5: 50%

Memoria: 60%

Yonathan Patricio:

Parte 1: 50%

Parte 2: 50%

Parte 3: 50%

Parte 4: 50%

Parte 5: 50%

Memoria: 40%

PRACTICA 5: DESPLIEGUE DE LA APLICACIÓN “FILTROS API”

1. Introducción.....	64
2. Diseño y Desarrollo de la Aplicación (parte 1)	64
2.1. Protocolo de Comunicación (Redis).....	64
2.2 Servidor de Procesamiento(servidor_proc.py):	65
2.3 Api REST y Base de Datos (api_flask.py)	66
2.4 Cliente CLI (cliente.py)	68
3. Despliegue e Infraestructura Docker (parte 2)	69
3.1. Contenerización	69
3.2. Proxy (Nginx).....	69
3.3 Ejercicio 1 Despliegue de la aplicacion Práctica 5:	69
4. Pruebas y Resultados	71
Prueba 1: Procesamiento Exitoso.....	71
Prueba 2: Interfaz Web:	72
Prueba 3: Lanzar varios trabajos a la vez:	73
Prueba 5: Manejo de Errores.	76
Ejercicio 2. Desplegar practica 4.	77
5.Conclusión.....	78
6.Participación.....	78

1. Introducción

En la Práctica 13 de Programación Avanzada hicimos un script para aplicar filtros a imágenes que utiliza procesos de cpu intensivos que calculan chunks para procesar estas imágenes y aplicarles el filtro mediante unas funciones. Hemos utilizado esto para crear el sistema.

En esta Práctica 5 de SD creamos un sistema para el procesamiento de imágenes aplicando filtros (sepia,blur,grises). Tenemos una API recibe las peticiones, un sistema de colas (Redis) las gestiona, y un servidor de procesamiento que ejecuta la lógica de la aplicación.

Hemos desplegado todo ello utilizando Docker y Docker Compose. Además, hemos integrado mejoras opcionales como una base de datos, autenticación y un cliente de línea de comandos (click).

2. Diseño y Desarrollo de la Aplicación (parte 1)

2.1. Protocolo de Comunicación (Redis)

Redis actúa como el intermediario entre la api y el servidor. Hemos utilizado la estructura (set) de listas de Redis para gestionar la cola de trabajos.

Funcionamiento:

Cuando la API recibe una petición, revisa que la imagen exista en la carpeta (/app/static). Guarda la ruta de la imagen en Redis en la clave {job_id}:input_ruta.

La API inserta el UUID del trabajo en la lista trabajos mediante el comando RPush.

El servidor recupera el UUID con blpop y usa la ruta almacenada en Redis para abrir la imagen desde la carpeta (/static).

Tras aplicar los filtros y guardar la imagen procesada en la carpeta (/static).El servidor guarda esta nueva ruta en la clave de Redis {job_id}:result

Redis almacena el progreso del procesamiento de cada trabajo gracias a que el servidor actualiza paso a paso cada vez que se procesa un “trozo” (chunk) de la imagen.

Para el estado de cada trabajo, hemos hecho esta estructura de claves en Redis, donde {job_id} es el UUID generado:

- {job_id}:status: Indica el estado actual (queued, processing, finished, failed).
- {job_id}:progress: Un entero de 0 a 100 indicando el porcentaje.
- {job_id}:input_ruta: Ruta de la imagen original.
- {job_id}:input_filtro: Tipo de filtro a aplicar (sepia, blur, grises).
- {job_id}:result: Contiene el nombre del archivo final y un mensaje de éxito/error

Ejemplo en redis-cli KEYS *

```
1) "5a6a48b1-d158-4e1f-8e29-64af60958d02:result"
2) "a23dc5ff-37e0-40d0-ad9d-b77fb5cc48f0:input_filtro"
3) "c663976e-d860-45c0-ae4d-dde704efa84c:progress"
4) "5a6a48b1-d158-4e1f-8e29-64af60958d02:input_filtro"
5) "a23dc5ff-37e0-40d0-ad9d-b77fb5cc48f0:result"
6) "c0d30cbf-5e52-4dd9-bf83-3e4da179c9ca:result"
7) "c0d30cbf-5e52-4dd9-bf83-3e4da179c9ca:input_ruta"
8) "c663976e-d860-45c0-ae4d-dde704efa84c:result"
9) "1934cdb9-00b3-4f52-bb8d-a7fd8d2b3229:input_ruta"
10) "1934cdb9-00b3-4f52-bb8d-a7fd8d2b3229:progress"
11) "ba4c2fa7-49bc-4da5-81ae-832bfb37e945:result"
12) "c663976e-d860-45c0-ae4d-dde704efa84c:input_ruta"
13) "ba4c2fa7-49bc-4da5-81ae-832bfb37e945:input_ruta"
14) "c0d30cbf-5e52-4dd9-bf83-3e4da179c9ca:progress"
15) "6e2c042f-a74a-41fb-b027-744eb50e8b3d:input_ruta"
16) "1934cdb9-00b3-4f52-bb8d-a7fd8d2b3229:status"
17) "c663976e-d860-45c0-ae4d-dde704efa84c:status"
18) "6e2c042f-a74a-41fb-b027-744eb50e8b3d:input_filtro"
```

Ejemplo de cómo el servidor va actualizando el progreso en redis.

```
127.0.0.1:6379> GET 4be4fa29-e3cc-430e-89e7-361621eb1e70:progress
"0"
127.0.0.1:6379> GET 4be4fa29-e3cc-430e-89e7-361621eb1e70:progress
"0"
127.0.0.1:6379> GET 4be4fa29-e3cc-430e-89e7-361621eb1e70:progress
"66"
127.0.0.1:6379> GET 4be4fa29-e3cc-430e-89e7-361621eb1e70:progress
"100"
```

2.2 Servidor de Procesamiento(servidor_proc.py):

Lógica (Funciones de filtros): El servidor implementa tres filtros de imagen utilizando matrices, escala de grises, efecto sepia y desenfoque (blur).

Paralelismo: Utilizamos ProcessPoolExecutor (multiprocessing). La imagen se divide en trozos (chunks) según el número de CPUs disponibles. Cada proceso trabaja en un trozo de la imagen de forma paralela.

Cada vez que termina un chunk se va actualizando el progreso en la clave {job_id}:progress en Redis.

El programa está en un bucle esperando que aparezcan trabajos en la cola de Redis. Esto bloquea el programa hasta que se hace la llamada a la API.

- redis_client.blpop('trabajos').

Cuando exista un trabajo, se creará un hilo o proceso que lo atienda.

- t = threading.Thread(target=ejecutar_trabajo, args=(job_id,))
- t.start()

El hilo obtiene de Redis la información necesaria para saber qué debe calcular.

Mientras calcula, cada vez que complete resultado se actualiza en Redis el porcentaje.

Dentro del bucle for future in as_completed(...), cada vez que un trozo de imagen termina de procesarse se actualiza Redis:

- completados += 1
- progreso = int((completados / num_chunks) * 100)
- redis_client.set(f'job_id:progress', progreso)

El servidor muestra por la terminal como se van procesando los trabajos, esto es lo que se va almacenando en Redis.

```
server_container | Job ec6f6232-54ee-4353-a05e-40a6c1d3baba: Progreso 33%
server_container | Job ec6f6232-54ee-4353-a05e-40a6c1d3baba: Progreso 66%
server_container | Job ec6f6232-54ee-4353-a05e-40a6c1d3baba: Progreso 100%
```

2.3 Api REST y Base de Datos (api_flask.py)

Crear un nuevo trabajo (POST /jobs) y consultar su estado (GET /jobs/<id>).

POST /Jobs/:

POST: Recibe una ruta de imagen y un filtro. Genera un UUID único, guarda los datos en Redis y envía el trabajo. Devuelve un JSON con los datos.

Ejemplo de una petición POST bien hecha:

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend/static$ http POST http://localhost/jobs ruta="static/mesi.png" filtro="sepia" --auth alumno:alumno_pass
HTTP/1.1 201 CREATED
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 193
Content-Type: application/json
Date: Sat, 13 Dec 2025 19:18:17 GMT
Server: nginx/1.29.3

{
  "input_filtro": "sepia",
  "input_ruta": "static/mesi.png",
  "job_id": "5a6a48b1-d158-4elf-8e29-64af60958d02",
  "status": "processing",
  "uri": "http://localhost/jobs/5a6a48b1-d158-4elf-8e29-64af60958d02"
}
```

GET /jobs/<id>:

Consulta el estado de un trabajo. Si el trabajo está en curso, devuelve el progreso. Si ha finalizado (status: finished), devuelve el resultado final.

La petición GET recupera status y progress. Si el status no es "finished", devuelve el JSON básico con el progreso actual.

Si el trabajo ha finalizado, el JSON contiene el resultado. Obtenemos el resultado y se procesa al JSON.

Ejemplo de una petición GET bien hecha que devuelve el JSON resultado procesado correctamente.

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend/static$ http GET http://localhost/jobs/5a6a48b1-d158-4e1f-8e29-64af60958d02 --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 282
Content-Type: application/json
Date: Sat, 13 Dec 2025 19:21:13 GMT
Server: nginx/1.29.3

{
  "input_filtro": "sepia",
  "input_ruta": "static/mesi.png",
  "job_id": "5a6a48b1-d158-4e1f-8e29-64af60958d02",
  "progress": "100",
  "result": {
    "archivo": "mesi_sepia.png",
    "mensaje": "Procesado correctamente"
  },
  "status": "finished",
  "uri": "http://localhost/jobs/5a6a48b1-d158-4e1f-8e29-64af60958d02"
}
```

Ejemplo de una petición GET que aún no ha sido procesada correctamente, aparecerá el progreso que lleva. Como aún no ha finalizado no aparece el “result” en el JSON.

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend$ http GET http://localhost/jobs/c663976e-d860-45c0-ae4d-dde704efa84c --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 207
Content-Type: application/json
Date: Sat, 13 Dec 2025 19:51:46 GMT
Server: nginx/1.29.3

{
  "input_filtro": "blur",
  "input_ruta": "static/cr7.png",
  "job_id": "c663976e-d860-45c0-ae4d-dde704efa84c",
  "progress": "33",
  "status": "processing",
  "uri": "http://localhost/jobs/c663976e-d860-45c0-ae4d-dde704efa84c"
}
```

Base de Datos MariaDB:

Como extra implementamos una base de datos P5_db en la que guardamos el registro de trabajos hechos. También implementamos una autenticación básica para acceder a los endpoints (alumno:alumno_pass).

```
MariaDB [P5_db]> SHOW TABLES;
+-----+
| Tables_in_P5_db |
+-----+
| trabajos        |
+-----+
1 row in set (0.006 sec)

MariaDB [P5_db]> SELECT * FROM trabajos;
Empty set (0.023 sec)

MariaDB [P5_db]> SELECT * FROM trabajos;
+-----+-----+-----+
| id                                     | input_ruta      | input_filtro |
+-----+-----+-----+
| 5a6a48b1-d158-4e1f-8e29-64af60958d02 | static/mesi.png | sepia        |
+-----+-----+-----+
```

GET jobs: Consulta el estado de los trabajos realizados guardados en la base de datos.

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend$ http GET http://localhost/jobs --auth alu
mno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 780
Content-Type: application/json
Date: Sat, 13 Dec 2025 19:31:36 GMT
Server: nginx/1.29.3

{
  "trabajos": [
    {
      "input_filtro": "grises",
      "input_ruta": "static/mesi.png",
      "job_id": "1934cdb9-00b3-4f52-bb8d-a7fd8d2b3229",
      "status": "finished",
      "uri": "http://localhost/jobs/1934cdb9-00b3-4f52-bb8d-a7fd8d2b3229"
    },
    {
      "input_filtro": "blur",
      "input_ruta": "static/mesi.png",
      "job_id": "33e40e78-aal7-4d0f-al42-ab3dc29bc4a4",
      "status": "finished",
      "uri": "http://localhost/jobs/33e40e78-aal7-4d0f-al42-ab3dc29bc4a4"
    }
  ]
}
```

2.4 Cliente CLI (cliente.py)

Creamos la herramienta de línea de comandos usando la librería click para facilitar la interacción con la API (new, status, list)

NEW: Enviar un nuevo trabajo.

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend$ python cliente.py new "static/doc
ker.png" --filtro grises
ID: af1e3272-3dc2-4b4c-a09a-f78f08be1936
Estado: processing
URI: http://localhost/jobs/af1e3272-3dc2-4b4c-a09a-f78f08be1936
```

STATUS: Consultar estado por ID.

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend$ python cliente.py status af1e3272-3dc2-4b
4c-a09a-f78f08be1936
Trabajo -> af1e3272-3dc2-4b4c-a09a-f78f08be1936
Estado: finished
Progreso: 100%
Ruta: static/docker.png
Resultado: docker_grises.png
```

LIST: Muestra todos los trabajos realizados

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend$ python cliente.py list
- ID: 15948880-fc69-4110-8165-8891646b4ca7 Estado: finished Filtro: sepia Ruta: sta
tic/cr7.png
- ID: 1934cdb9-00b3-4f52-bb8d-a7fd8d2b3229 Estado: desconocido Filtro: grises Ruta:
static/mesi.png
- ID: 33e40e78-aal7-4d0f-al42-ab3dc29bc4a4 Estado: desconocido Filtro: blur Ruta: s
tatic/mesi.png
- ID: 37f7f3ec-5238-4ba5-b202-3c322a1afba3 Estado: finished Filtro: blur Ruta: stat
ic/cr7.png
- ID: 41467e9d-b298-4fdf-b3bf-f475affb390a Estado: finished Filtro: grises Ruta: st
atic/cr7.png
- ID: 4be4fa29-e3cc-430e-89e7-361621eb1e70 Estado: finished Filtro: blur Ruta: stat
ic/cr7.png
- ID: 4c721d03-74b0-4073-bbff-8f3b79960c45 Estado: finished Filtro: sepia Ruta: sta
tic/cr7.png
```

3. Despliegue e Infraestructura Docker (parte 2)

3.1. Contenerización

Hemos creado dos imágenes personalizadas:

Dockerfile_servidor: Basada en Python, instala redis, numpy y Pillow. Copia el script del servidor.

Dockerfile_api: Basada en Python, instala Flask, gunicorn, redis, SQLAlchemy. Ejecuta la API con Gunicorn usando 4 workers.

Utilizan variables de entorno (REDIS_HOST, SQLALCHEMY_DATABASE_URI)

3.2. Proxy (Nginx)

Nginx escucha en el puerto 80 y redirige el tráfico al contenedor de la API (proxy_pass http://api_container:5000).

3.3 Ejercicio 1 Despliegue de la aplicación Práctica 5:

Creemos la carpeta compose-ej1 (github: [./P5/backend/compose-ej1](#)) con el archivo docker-compose.yml para el despliegue.

Para los servicios API y SERVER le indicamos a docker compose que debe construirlas. Dándole el contexto de que tenemos dos ficheros Dockerfile_api y Dockerfile_server creados.

Base de datos: ../basedatos:/var/lib/mysql para asegurar el servicio mariadb.

Creemos el fichero docker-compose.yml define 5 servicios:

- Redis: imagen oficial redis.
- MariaDB Imagen oficial, con persistencia de datos mediante volumen en ../basedatos.
- Server: Construido desde Dockerfile_servidor. Reinicio automáticamente
- API: Construido desde Dockerfile_API.
- Nginx: Proxy

Archivos imágenes: Un problema que tuvimos fue compartir las imágenes entra el cliente, la api y el servidor. Para ello montamos el volumen ../backend/static:/app/static en la api y el server que permite que las imágenes sean accesibles dentro de la aplicación.

Dependencias: Definimos depends_on Redis para asegurar que la cola Redis esté disponible antes de iniciar la api y el server también configuramos restart: on-failure para que los contenedores se reiniciaran automáticamente si unos se conectaban antes que otros.

Desplegamos todo con docker compose up.

Figura 1: Salida del comando docker compose ps.

```
uo300480@sd-300480:~/sd-pl2-g14/P5/compose-ej1$ docker compose ps
WARN[0000] /home/uo300480/sd-pl2-g14/P5/compose-ej1/docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
api_container        compose-ej1-api      "gunicorn -b 0.0.0.0..." api         8 minutes ago  Up 7 minutes  5000/tcp
mariadb_p5_db        mariadb:latest       "docker-entrypoint.s..." mariadb     8 minutes ago  Up 8 minutes  3306/tcp
nginx_container      nginx                "/docker-entrypoint..." nginx       8 minutes ago  Up 2 minutes  0.0.0.0:80->80/tcp, [::]:80->80/tcp
redis_container      redis                "docker-entrypoint.s..." redis      14 hours ago   Up 8 minutes  0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
server_container     compose-ej1-server   "python -u servidor_..." server      8 minutes ago  Up 8 minutes
```

Figura 2: Salida de docker network ls.

```
uo300480@sd-300480:~/sd-pl2-g14/P5/compose-ej1$ docker network ls
NETWORK ID          NAME                DRIVER            SCOPE
3d2a42fae478        bridge             bridge            local
ad3d018b9e64        compose-ej1_default bridge            local
65a0888fe3bb        host              host              local
d751824c9468        none              null              local
06b4cfd7247a        pruebas           bridge            local
```

4. Pruebas y Resultados

Prueba 1: Procesamiento Exitoso.

Lanzamos un trabajo, consultando el progreso y obteniendo el resultado final. Utilizamos como imagen inicial esta:



Realizamos una petición POST:

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend/static$ http POST http://localhost/jobs ru
ta="static/mesi.png" filtro="sepia" --auth alumno:alumno_pass
HTTP/1.1 201 CREATED
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 193
Content-Type: application/json
Date: Sat, 13 Dec 2025 19:18:17 GMT
Server: nginx/1.29.3

{
  "input_filtro": "sepia",
  "input_ruta": "static/mesi.png",
  "job_id": "5a6a48b1-d158-4elf-8e29-64af60958d02",
  "status": "processing",
  "uri": "http://localhost/jobs/5a6a48b1-d158-4elf-8e29-64af60958d02"
}
```

Petición GET:

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend/static$ http GET http://localhost/jobs/5a6
a48b1-d158-4elf-8e29-64af60958d02 --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 282
Content-Type: application/json
Date: Sat, 13 Dec 2025 19:21:13 GMT
Server: nginx/1.29.3

{
  "input_filtro": "sepia",
  "input_ruta": "static/mesi.png",
  "job_id": "5a6a48b1-d158-4elf-8e29-64af60958d02",
  "progress": "100",
  "result": {
    "archivo": "mesi_sepia.png",
    "mensaje": "Procesado correctamente"
  },
  "status": "finished",
  "uri": "http://localhost/jobs/5a6a48b1-d158-4elf-8e29-64af60958d02"
}
```

El servidor procesó la imagen correctamente y le aplicó el filtro dado.



Si consultamos la base de datos vemos el trabajo guardado:

```
MariaDB [P5_db]> SHOW TABLES;
+-----+
| Tables_in_P5_db |
+-----+
| trabajos         |
+-----+
1 row in set (0.006 sec)

MariaDB [P5_db]> SELECT * FROM trabajos;
Empty set (0.023 sec)

MariaDB [P5_db]> SELECT * FROM trabajos;
+-----+-----+-----+
| id                | input_ruta      | input_filtro |
+-----+-----+-----+
| 5a6a48b1-d158-4e1f-8e29-64af60958d02 | static/mesi.png | sepia        |
+-----+-----+-----+
```

Prueba 2: Interfaz Web:

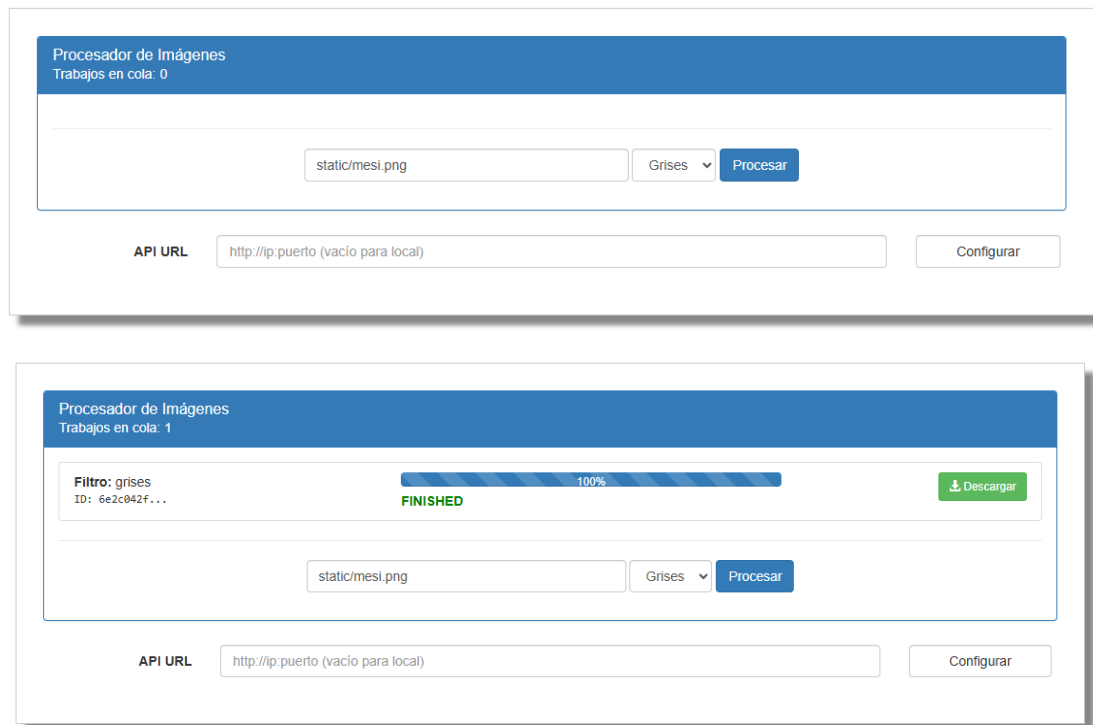
Hemos creado un interfaz para tenerlo de forma más visual para el video reciclando la usada en la practica 4 pero con algunos cambios.

Tenemos un proceso de autenticación en la que debemos de introducir el usuario alumno y la contraseña alumno_pass.

The screenshot shows a web form with the following elements:

- API URL:** A text input field containing "http://ip.puerto (vacío para local)" and a "Configurar" button to its right.
- Usuario:** A text input field containing the value "alumno".
- Contraseña:** A password input field containing "*****" with a toggle icon (an eye) on the right.
- Entrar:** A button located below the password field.

Saldrá una barra en la que tenemos que introducir la ruta de la imagen que debería de estar en la carpeta static para que funcione. Hay que seleccionar el filtro que queramos utilizar. Cuando le demos al botón Procesar se verá una barra que se va actualizando poco a poco el progreso hasta llegar a 100 (finalizado) y podremos descargar la imagen con el filtro ya aplicado.



Prueba 3: Lanzar varios trabajos a la vez:

Lanzamos dos peticiones a la vez, se puede ver que uno de ellos está siendo procesado y el otro aparece en cola (queued) se crean dos hilos para procesar las imágenes.

```
(flaskenv) uo300480@sd-300480:~/sd-pl2-g14/P5/compose-ej1$ http POST http://localhost/jobs ruta="static/docker.png" filtro="blur" --auth alumno:alumno_pass
HTTP/1.1 201 CREATED
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 194
Content-Type: application/json
Date: Thu, 18 Dec 2025 22:22:22 GMT
Server: nginx/1.29.3

{
  "input_filtro": "blur",
  "input_ruta": "static/docker.png",
  "job_id": "665164c6-3408-4be6-9621-ea49bf296d9d",
  "status": "processing",
  "uri": "http://localhost/jobs/665164c6-3408-4be6-9621-ea49bf296d9d"
}
```

```
(flaskenv) uo300480@sd-300480:~/sd-pl2-g14/P5/compose-ej1$ http POST http://localhost/jobs ruta="static/docker.png" filtro="grises" --auth alumno:alumno_pass
HTTP/1.1 201 CREATED
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 192
Content-Type: application/json
Date: Thu, 18 Dec 2025 22:22:30 GMT
Server: nginx/1.29.3

{
  "input_filtro": "grises",
  "input_ruta": "static/docker.png",
  "job_id": "f7cdefd7-c6d4-4ae9-9309-ef487e96b650",
  "status": "queued",
  "uri": "http://localhost/jobs/f7cdefd7-c6d4-4ae9-9309-ef487e96b650"
}
```

Si hacemos GET a cada uno de los trabajos vemos que se están procesando.

```
(flaskenv) uo300480@sd-300480:~/sd-pl2-g14/P5/compose-ej1$ http GET http://localhost/jobs/f7cdefd7-c6d4-4ae9-9309-ef487e96b650 --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 212
Content-Type: application/json
Date: Thu, 18 Dec 2025 22:22:42 GMT
Server: nginx/1.29.3

{
  "input_filtro": "grises",
  "input_ruta": "static/docker.png",
  "job_id": "f7cdefd7-c6d4-4ae9-9309-ef487e96b650",
  "progress": "66",
  "status": "processing",
  "uri": "http://localhost/jobs/f7cdefd7-c6d4-4ae9-9309-ef487e96b650"
}
```

```
(flaskenv) uo300480@sd-300480:~/sd-pl2-g14/P5/compose-ej1$ http GET http://localhost/jobs/665164c6-3408-4be6-9621-ea49bf296d9d --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 210
Content-Type: application/json
Date: Thu, 18 Dec 2025 22:22:57 GMT
Server: nginx/1.29.3

{
  "input_filtro": "blur",
  "input_ruta": "static/docker.png",
  "job_id": "665164c6-3408-4be6-9621-ea49bf296d9d",
  "progress": "66",
  "status": "processing",
  "uri": "http://localhost/jobs/665164c6-3408-4be6-9621-ea49bf296d9d"
}
```

Si volvemos a hacer un GET vemos que ha sido procesado correctamente y se crea el mensaje final “Procesado correctamente” con el nombre de la imagen procesada.

```
(flaskenv) uo300480@sd-300480:~/sd-pl2-g14/P5/compose-ej1$ http GET http://localhost/jobs/665164c6-3408-4be6-9621-ea49bf296d9d --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 284
Content-Type: application/json
Date: Thu, 18 Dec 2025 22:28:08 GMT
Server: nginx/1.29.3

{
  "input_filtro": "blur",
  "input_ruta": "static/docker.png",
  "job_id": "665164c6-3408-4be6-9621-ea49bf296d9d",
  "progress": "100",
  "result": {
    "archivo": "docker_blur.png",
    "mensaje": "Procesado correctamente"
  },
  "status": "finished",
  "uri": "http://localhost/jobs/665164c6-3408-4be6-9621-ea49bf296d9d"
}
```

Si consultamos las KEYS en redis-cli y hacemos un GET del resultado nos da la imagen final procesada.

```
127.0.0.1:6379> KEYS *
1) "f7cdefd7-c6d4-4ae9-9309-ef487e96b650:status"
2) "665164c6-3408-4be6-9621-ea49bf296d9d:progress"
3) "f7cdefd7-c6d4-4ae9-9309-ef487e96b650:input_ruta"
4) "665164c6-3408-4be6-9621-ea49bf296d9d:input_filtro"
5) "4269c5f6-d4c5-405e-9ffb-281e23ea985d:progress"
6) "665164c6-3408-4be6-9621-ea49bf296d9d:status"
7) "4269c5f6-d4c5-405e-9ffb-281e23ea985d:input_filtro"
8) "4269c5f6-d4c5-405e-9ffb-281e23ea985d:status"
9) "665164c6-3408-4be6-9621-ea49bf296d9d:input_ruta"
10) "f7cdefd7-c6d4-4ae9-9309-ef487e96b650:input_filtro"
11) "4269c5f6-d4c5-405e-9ffb-281e23ea985d:result"
12) "f7cdefd7-c6d4-4ae9-9309-ef487e96b650:result"
13) "f7cdefd7-c6d4-4ae9-9309-ef487e96b650:progress"
14) "665164c6-3408-4be6-9621-ea49bf296d9d:result"
15) "4269c5f6-d4c5-405e-9ffb-281e23ea985d:input_ruta"
127.0.0.1:6379> GET 4269c5f6-d4c5-405e-9ffb-281e23ea985d:result
"docker_sepia.png;Procesado correctamente"
127.0.0.1:6379> GET 665164c6-3408-4be6-9621-ea49bf296d9d:result
"docker_blur.png;Procesado correctamente"
127.0.0.1:6379>
```

Obtenemos la imagen final con el filtro aplicado guardada en la carpeta static.



Vemos que en la base de datos siguen existiendo los trabajos, aunque reiniciemos los contenedores.

```
MariaDB [P5_db]> SELECT * FROM trabajos;
+-----+-----+-----+
| id                | input_ruta          | input_filtro |
+-----+-----+-----+
| 4269c5f6-d4c5-405e-9ffb-281e23ea985d | static/docker.png | sepia        |
| 665164c6-3408-4be6-9621-ea49bf296d9d | static/docker.png | blur         |
| f7cdefd7-c6d4-4ae9-9309-ef487e96b650 | static/docker.png | grises       |
+-----+-----+-----+
3 rows in set (0.003 sec)
```

Prueba 5: Manejo de Errores.

Error en la imagen no se ha procesado el filtro correctamente:

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend$ http GET http://localhost/jobs/ec6f6232-54ee-4353-a05e-40a6c1d3baba --auth alumno:alumno_pass
HTTP/1.1 200 OK
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 245
Content-Type: application/json
Date: Sat, 13 Dec 2025 22:16:54 GMT
Server: nginx/1.29.3

{
  "error": "Error procesando la imagen",
  "input_filtro": "sepia",
  "input_ruta": "static/docker.png",
  "job_id": "ec6f6232-54ee-4353-a05e-40a6c1d3baba",
  "progress": "100",
  "status": "failed",
  "uri": "http://localhost/jobs/ec6f6232-54ee-4353-a05e-40a6c1d3baba"
}
```

Consultamos esta imagen en redis haciendo un GET al mismo UUID. Vemos que en el estado sale que ha fallado el proceso.

```
127.0.0.1:6379> GET ec6f6232-54ee-4353-a05e-40a6c1d3baba:result
""
127.0.0.1:6379> GET ec6f6232-54ee-4353-a05e-40a6c1d3baba:status
"failed"
127.0.0.1:6379> GET ec6f6232-54ee-4353-a05e-40a6c1d3baba:input_ruta
"static/docker.png"
127.0.0.1:6379> GET ec6f6232-54ee-4353-a05e-40a6c1d3baba:progress
"100"
```

Realizamos una petición POST con la ruta incorrecta (error 400):

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend$ http POST http://localhost/jobs/ruta="static/doer.png" filtro="sepia" --auth alumno:alumno_pass
HTTP/1.1 400 BAD REQUEST
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 84
Content-Type: application/json
Date: Sat, 13 Dec 2025 22:13:57 GMT
Server: nginx/1.29.3

{
  "error": "Solicitud incorrecta se requiere que la ruta y el filtro sean correctos"
}
```

Ahora vamos a hacer un GET a un UUID que no existe (error 404):

```
(flaskenv) uo300480@sd-300480:~/P5-SD/backend$ http GET http://localhost/jobs/45353 --auth alumno:alumno_pass
HTTP/1.1 404 NOT FOUND
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Length: 32
Content-Type: application/json
Date: Sat, 13 Dec 2025 22:14:57 GMT
Server: nginx/1.29.3

{
  "error": "Trabajo inexistente"
}
```

Ejercicio 2. Desplegar practica 4.

Creamos la carpeta compose-ej2 dentro del Directorio de la práctica ./P5. Hicimos un docker-compose para el despliegue de la aplicación Lista de Tareas desarrollada en la Práctica 4.

Hemos configurado el archivo docker-compose.yml utilizando rutas relativas para hacer referencia a la Práctica 4 (código fuente, configuración de Nginx y base de datos). Así conseguimos que no se dupliquen archivos y que se reutilice la base de datos que teníamos creada.

Además, hemos implementado como en el anterior despliegue `depends_on` para asegurar el orden correcto de arranque: Nginx espera a la API, y la API espera a la base de datos.

Lanzamos el servicio con `docker compose up -d`:

Terminal con `docker compose ps`: Muestra los 3 contenedores (`mariadb-p4`, `flask-app-p4`, `nginx-p4`) en estado "Up".

```
uo301402@sd-uo301402:~/S5/compose-ej2$ docker compose ps
WARN[0000] /home/uo301402/S5/compose-ej2/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                STATUS                        PORTS                    COMMAND                                SERVICE      CREATED
mariadb_container   Up 8 minutes                 3306/tcp                "docker-entrypoint.s..."            mariadb      8 minutes ago
nginx_container     Up 8 minutes                 0.0.0.0:80->80/tcp, [::]:80->80/tcp  "/docker-entrypoint..."            nginx        8 minutes ago
tarea_app_flask     Up 8 minutes                 5000/tcp                "gunicorn -b 0.0.0.0..."            tarea_app_flask 8 minutes ago
```

Terminal con `docker network ls`: Señala la red que se ha creado (`compose-ej2`).

```
uo301402@sd-uo301402:~/S5/compose-ej2$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
21c989263234        bridge             bridge              local
bfb8f2be2f80        compose-ej1_default bridge              local
8e9cd38722d8        compose-ej2_default bridge              local
```

Accedimos al navegador a través de `http://localhost...` y verificamos que la aplicación funciona correctamente:

Servidor Cargar

Deja en blanco para usar como servidor REST el mismo que sirvió esta página.

Escribe `http://ip:puerto` para conectar con otro servidor via REST

Usuario

Contraseña

Entrar

5.Conclusión.

En esta práctica, creando un sistema distribuido, hemos mejorado nuestras habilidades con tecnologías como Redis, API Flask y Docker.

Docker Compose ha ayudado muchísimo respecto a la práctica anterior. Nos ha facilitado el despliegue de la aplicación, permitiéndonos sustituir el script por un archivo (docker-compose.yml) que define toda la infraestructura.

Además, hemos aprendido conceptos como la persistencia y la comunicación. El uso de volúmenes con rutas relativas ha sido fundamental ya que ha sido uno de los problemas que nos hemos encontrado durante el desarrollo y hemos solucionado. Finalmente, hemos aprendido sobre las redes de Docker, que simplifican la conexión entre la API, Redis y la base de datos (MariaDB) utilizando solo los nombres de servicio.

6.Participación.

Ambos hemos colaborado en el desarrollo del api y del servidor también en la resolución de los problemas de despliegue y en la redacción de la memoria.

Adrián Manso: 50%

Yonathan Patricio: 50%